제 4장 그리디 알고리즘

그리디(Greedy) 알고리즘 개요

그리디 알고리즘이란?

❖제약조건에서 목표를 달성하기 위한 최적화 문제를 해결하는 알고리즘 중 하나.

- 최적화 (optimization) 문제: 가능한 해들 중에서 가장 좋은 (최대 또는 최소) 해를 찾는 문제.
- 욕심쟁이 방법, 탐욕적 방법, 탐욕 알고리즘 등으로 불림.

❖문제를 단계적으로 해결하면서, 한 단계마다 그 순간의 최선의 solution 이라고 생각되는 해(solution)을 선택해서 최종적인 해(solution)을 찾아가는 알고리즘 전략

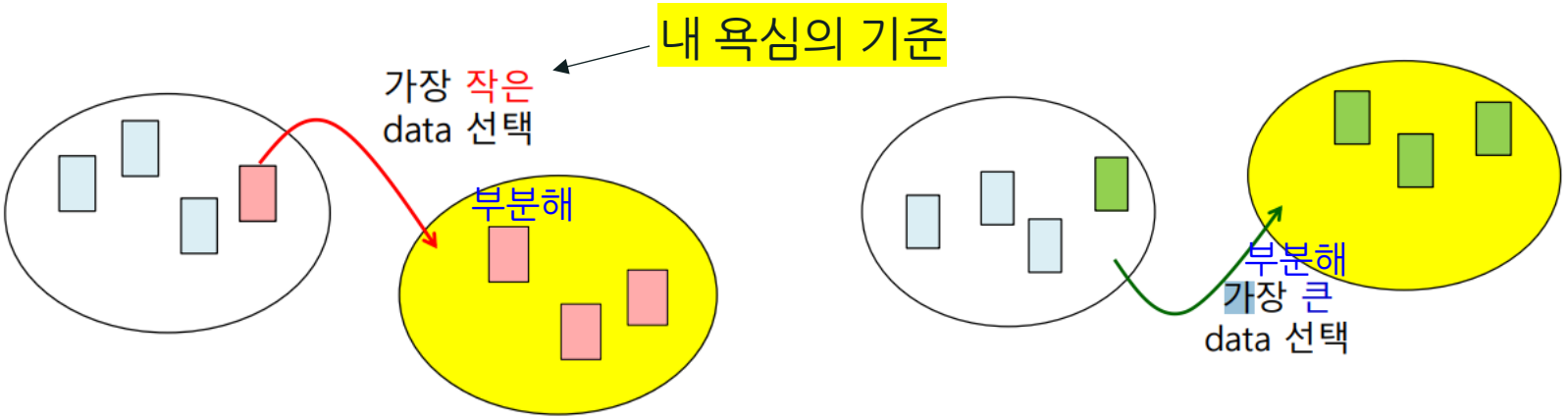
- 단계마다 반복적으로 해를 선택하는 방법 : (입력) 데이터 간의 관계를 고려하지 않고 수행 과정에서 현 상황에서 가장 좋은 (locally optimal) '욕심내어' (최소 또는 최대) 것을 가진 데이터를 선택.
- 매 단계마다 전체적으로 최적인지는 판단하지 않고 현재 최적이라고 판단되는 결정을 함. 즉 과거(이전 선택)나 미래(앞으로의 선택)는 생각하지 않고, 오직 현재의 최대 만족만을 추구.
- 이러한 선택을 '**근시안적**'(近視眼)인 선택이라고 말함.

그리디(Greedy) 알고리즘 개요

그리디 알고리즘이란?

❖ 즉, 그리디 알고리즘은 근시안적인 선택으로 부분적인 최선해를 찾고, 이들을 모아서 문제의 최적해를 얻음.

- ➢ 일단 한번 선택하면 이를 절대로 번복하지 않는다. 즉, 선택한 데이터를 버리고 다른 것을 취하지 않는다.
- ➢ 이러한 특성 때문에 대부분의 그리디 알고리즘들은 매우 단순하며, 소수의 문제들만이 그리디 알고리즘으로 해결된다.



<그리디 알고리즘 수행 과정>

그리디(Greedy) 알고리즘 개요

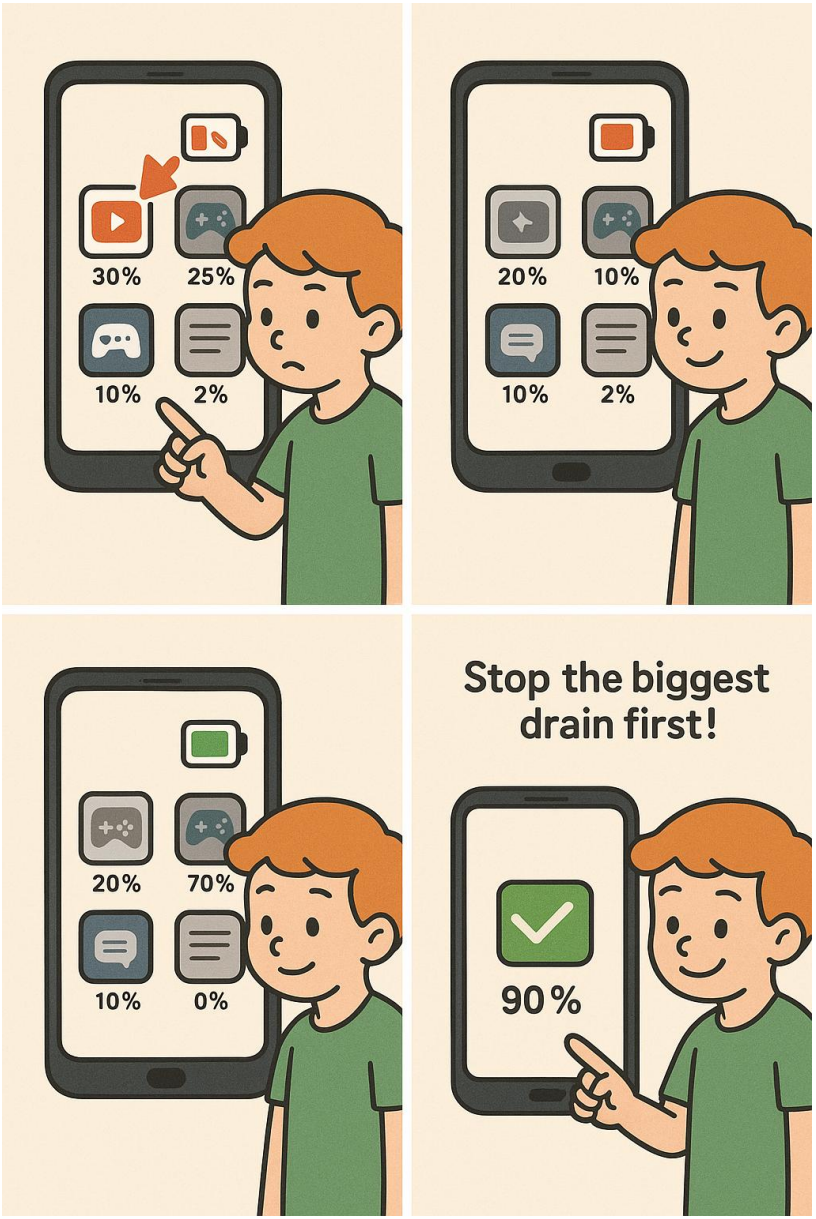
Cryptography & Information Security Laboratory

❖ 예) 밥상위에 다양한 반찬들 중 내가 가장 좋아하는 반찬들 부터 먹는 순서



사진 출처 : https://www.nongupin.co.kr/news/articleView.html?idxno=98669

그리디(Greedy) 알고리즘 개요



- **문제 상황**: 여러 개의 앱이 켜져 있고, 배터리가 부족하다.
- **그리디 접근**: 배터리를 가장 많이 소모하는 앱부터 종료한다.
- **직관**: 매 순간 가장 효과적으로 배터리를 절약하는 방법.

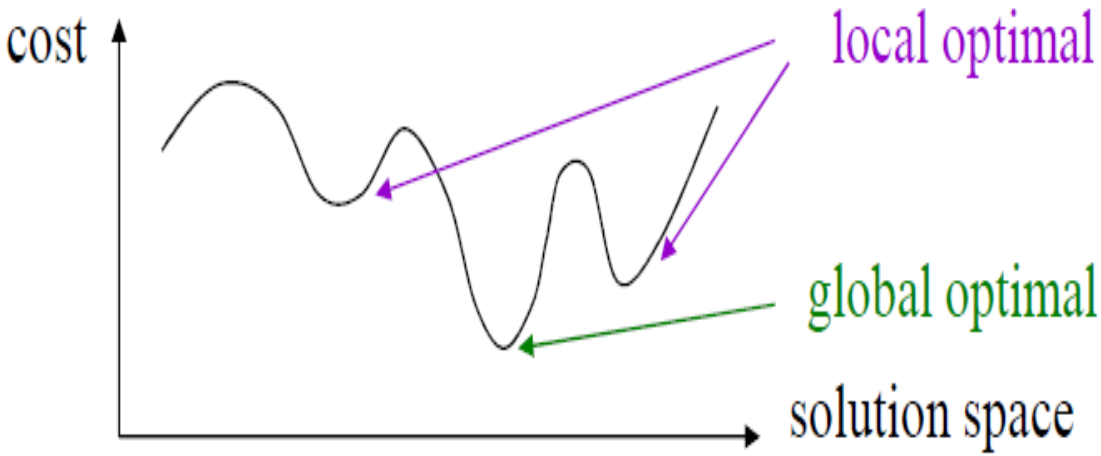
그리디(Greedy) 알고리즘 개요

그리디 알고리즘이란?

❖ Local Optimum and Global Optimum Solution

- local optimum : 어떤 단계에서 보이는 최선의 solution
- global optimum : 전체 문제에 대한 최적의 solution

- 하지만 모든 문제가 local optimum을 통해 global optimum을 만들 수 있지는 않음
- 어떤 알고리즘이 항상 global optimum 을 얻는다는 것을 주장하기 위해서는 논리적인 증명이 필요함.
  - ✓ 만일 증명을 할 수 없다면 알고리즘의 해는 suboptimal 일 수도 있으며, 이 경우 이것을 heuristic (휴리스틱) 알고리즘이라고 함.



그리디(Greedy) 알고리즘 개요

그리디 알고리즘으로 해결할 수 있는 문제는? (그리디 알고리즘 적용조건)

❖ **대부분 탐욕스런 선택 조건(greedy choice property)과 최적 부분 구조 조건(optimal substructure)이라는 두 가지 조건이 만족됨**

(1) 탐욕스런 선택 조건: 각 단계에서 선택한 "국소 최적해(local optimum)"가 전체 최적해(global optimum)로 이어질 수 있어야 한다는 조건.

→ 즉, 한 번의 선택이 이후 최적해를 만드는 데 모순이 되지 않아야 함.

- ➢ 예시 (성공):
  - 회의실 배정 문제: 가장 빨리 끝나는 회의를 고르는 선택이, 전체 회의 개수를 최대로 만드는 최적해와 일치.
  - 최소 신장 트리(MST): 크루스칼/프림 알고리즘에서, 매 단계 가장 짧은 간선을 고르는 선택이 전체 최적 MST로 이어짐.
- ➢ 예시 (실패):
  - 0-1 배낭 문제: "무게 대비 가치가 높은 물건"만 고르면, 전체 최적해를 보장하지 못함.
  - 거스름돈 문제 (1원, 3원): 6원을 거슬러 줄 때, 3원+3원이 최적해인데, 그리디는 3원+1원+1원+1원으로 suboptimal.

(2) 최적 부분 구조 조건: 문제(problem)의 최적해가 작은 부문제(subproblem)들의 최적해로부터 효율적으로 만들어질 수 있어야 함

- ➢ 예시 (성공):
  - 최단 경로 문제(Dijkstra): 출발점→목적지 최단 경로는 중간 지점까지의 최단 경로 + 중간→목적지 최단 경로로 분해 가능.MST: 어떤 서브그래프의 최적 MST를 구한 뒤, 이를 확장해도 전체 그래프의 MST 최적성이 유지됨.
- ➢ 예시 (실패): 외판원 문제(TSP): 최적해(최소 비용 순회)는 단순히 부분 최적 경로들의 합으로 표현되지 않음 → 그리디 적용 불가.

❖ 이러한 조건이 성립하지 않는 경우에는 탐욕 알고리즘은 최적해를 구하지 못함. 하지만, 이러한 경우에도 탐욕 알고리즘은 근사 알고리즘으로 사용 가능

❖ 대부분의 경우 계산 속도가 빠르기 때문에 실용적으로 사용가능 (전체를 고려치 않고 현재만을 생각하므로 비교적 간단히 구현)

❖ 실제로 사용하려면, 이 경우 역시 어느 정도까지 최적해에 가까운 해를 구할 수 있는지를 보장하려면 엄밀한 증명이 필요

그리디(Greedy) 알고리즘 개요

Cryptography & Information Security Laboratory

❖ 요약

- ➢ 그리디 알고리즘은, 일련의 단계를 거치면서 (최적)해를 구성한다고 판단되는 요소들을 모음
- ➢ 매 단계마다 전체적으로 최적인지는 판단하지 않고 현재 최선이라고 판단되는 결정을 함. 즉 과거나 미래는 생각하지 않고, 오직 현재의 최대 만족만을 추구(근시안)
- ➢ 전체를 고려치 않고 현재만을 생각하므로 비교적 간단히 구현
- ➢ 한번 내린 결정은 취소 • 변경하지 않기 때문에 빠른 계산 속도로 실용적으로 사용가능

❖ 그리디 알고리즘을 사용하려면?

- ➢ 최적화 문제 ∧ 최적 부분구조 ∧ 최적 선택임을 증명

1. 최적화 문제(optimization problem)
   - 가능한 모든 대안 중에서 가장 좋은 해답(최적해)을 고르는 문제
   - 가장 가까운 길을 찾아라, 최소 비용 신장 트리를 구하라, …
2. 최적 부분구조(optimal substructure)
   - 문제(problem)의 최적해가 부문제(subproblem)들의 최적해로부터 효율적으로 생성
   - 부문제는 원래의 문제와 똑같은 성질을 가지면서 단지 크기만 작은 문제
3. 최적 선택임을 증명
   - 각 단계에서 내리는 선택이 최적이라는 사실을 **수학적 귀납법**으로 증명

4.1 동전 거스름돈 문제

❖동전 거스름돈의 최소 동전 수를 찾는 그리디 알고리즘

- (달성 목표: 최소 동전 수, 제한조건 : 거스름돈 가치와 같도록)

❖동전 거스름돈(Coin Change)을 결정하는 해결 문제

- 매 순간마다 남은 거스름돈의 총액을 초과하지 않는 조건하에 'greedy, 욕심내어' 가장 큰 액면의 동전을 취함 (단, 동전의 액면은 500원, 100원, 50원, 10원, 1원)
- 가장 큰 액면을 가진 동전부터 차례로 선택하여 남은 거스름돈 액수보다 작을 경우 해답에 포함시킴

CoinChange Problem
입력: 거스름돈 액수 W
출력: 거스름돈 액수에 대한 최소 동전 수

```
1.    change=W, n500=n100=n50=n10=n1=0
                  // n500, n100, n50, n10, n1은 각각의 동전 카운트
2.    while ( change ≥ 500 )                  // 근시안 적인 선택 !!
        change = change-500, n500++           // 500원짜리 동전 수를 1 증가
3.    while ( change ≥ 100 )
        change = change-100, n100++           // 100원짜리 동전 수를 1 증가
4.    while ( change ≥ 50 )
        change = change-50, n50++             // 50원짜리 동전 수를 1 증가
5.    while ( change ≥ 10 )
        change = change-10, n10++             // 10원짜리 동전 수를 1 증가
6.    while ( change ≥ 1 )
        change = change-1, n1++               // 1원짜리 동전 수를 1 증가
7.    return (n500+n100+n50+n10+n1)                  // 총 동전 수를 리턴한다.
```

4.1 동전 거스름돈 문제

❖ 앞의 알고리즘은 남아있는 거스름 돈인 change에 대해 가장 높은 액면의 동전을 거스른다.

- ➢ 그리디 알고리즘의 근시안적인 특성 : 500원짜리 동전을 처리하는 과정에서 100원짜리, 50원짜리, 10원짜리, 1원짜리 동전을 몇 개 씩 거슬러 주어야 할 것인지에 대해서는 전혀 고려하지 않는다.

❖ **Is this greedy solution optimal ?** **Depends on the coin system**

❖ Ex) CoinChange 알고리즘이 최적으로 해를 주지않은 예 :

- ➢ 만약 동전 거스름돈 문제에서, **160원짜리 새로운 동전**의 종류가 추가된다면, CoinChange 알고리즘이 항상 최소 동전 수를 계산할 수 있을까?
  - 거스름돈이 200원이라면,
  - CoinChange 알고리즘은 160원짜리 동전 1개와 10원짜리 동전 4개로서 총 5개를 리턴한다.



CoinChange 알고리즘의 결과



최소 동전의 거스름돈

- ➢ CoinChange 알고리즘은 항상 최적의 답을 주는 것은 아니다

❖ 5 장에서는 어떤 경우에도 최적해를 찾는 동전 거스름돈을 위한 동적 계획 알고리즘을 소개한다.

4.1 동전 거스름돈 문제

❖ (예) 7원짜리 동전과 10원, 1원짜리 동전을 가지고있을 때, 거스름돈 14원을 내주어야 한다면
- ➢ Greedy Algorithm 이용하면 10원짜리 1개와 1원짜리 4개를 내어줌
- ➢ 최소 갯수의 동전으로 거스름 돈을 주는 방법은 7원짜리 2개를 내어줌

❖ **이 탐욕 알고리즘으로 거스름돈 교환 문제를 풀게 된다면**, **큰 단위의 동전이 작은 단위의 동전의 배수**가 되어야 탐욕 알고리즘을 적용할 수 있음.
- ➢ 위의 반례같은 경우는 7원은 1원의 배수지만, 10원이 7원의 배수가 되진 않기때문에 적용 불가능
- ➢ 실생활속의 우리 동전도 각각의 거스름돈이 서로의 배수/약수가 되기 때문에 적용 가능

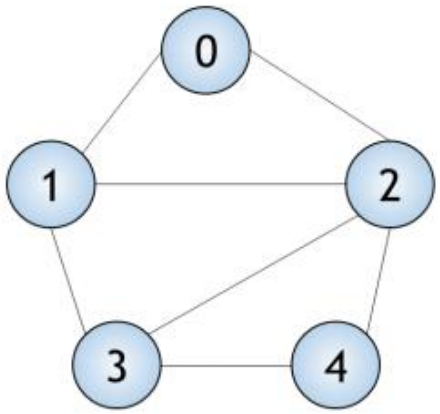
❖ **따라서, 유의할 점은** 매 순간마다 최선의 선택을 하여 나온 해가 반드시 전체적인 최적해는 아님.

❖ 그렇기 때문에, 탐욕 알고리즘을 사용할 때에는 전체적인 최적해 인지를 항상 염두할 필요가 있음.
- ➢ 즉, 탐욕 알고리즘을 사용하시기 전에 탐욕 알고리즘을 사용해도 되는지 검증 단계를 거쳐야 함.

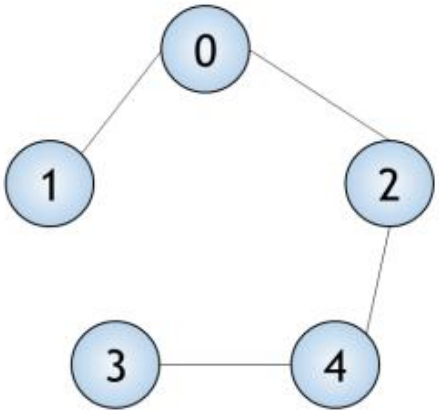
4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖ 신장 트리

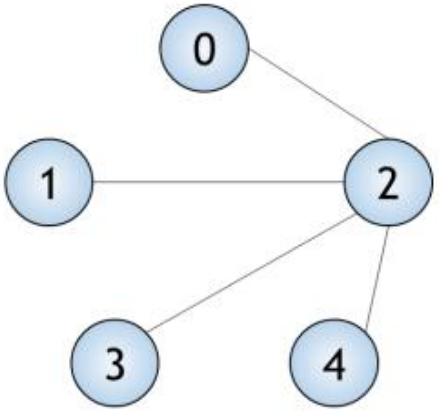
- **주어진 연결 그래프에서 사이클을 형성하는 간선을 제거하여 만든 트리**
- 정점의 수가 n 개이면 신장 트리의 간선 수는 n-1 개
- 계층 구조가 없는 그래프(사이클 없이 연결되어 있다는 트리 특성을 가짐)
- 활용 예) 각 정점이 도시를 나타내고 간선이 도로를 나타낸다면 신장 트리는 모든 도시들을 연결하는 최소한의 도로망



그래프



신장트리 1

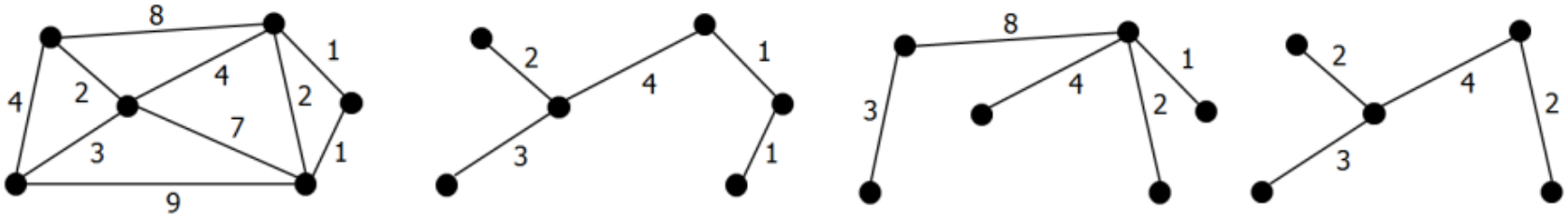


신장트리 2

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖ **최소 신장 트리** :

➢ 주어진 가중치 그래프에서 사이클이 없이 모든 점들을 연결시킨 트리들 중 선분(간선)들의 가중치 합이 최소인 트리



(a) (b) (c) (d)

(b) 최소 신장 트리, Cost =11

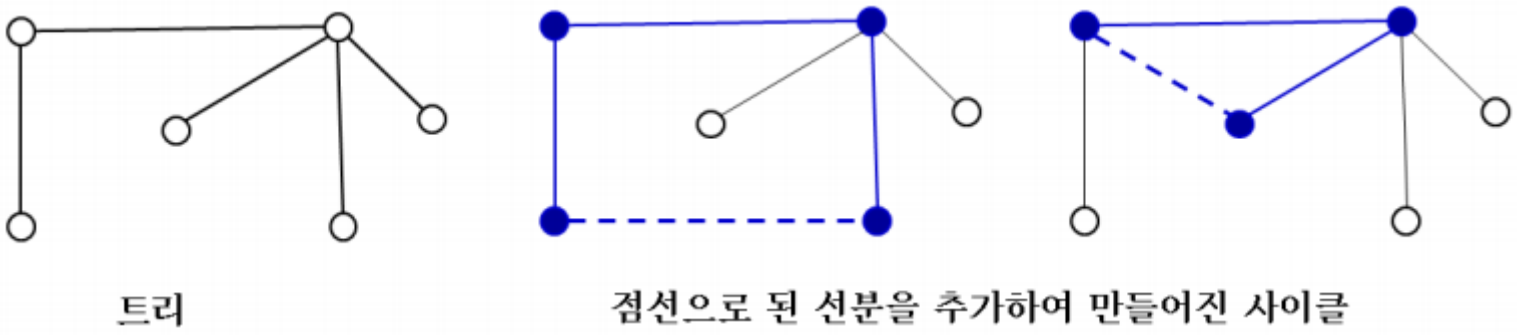
❖ (a)주어진 가중치 그래프, (b) 최소 신장 트리 (c),(d)는 최소 신장 트리 아님

❖ (c)는 가중치의 합이 (b)보다 크고, (d)는 트리가 주어진 그래프의 모든 노드를 포함하지 않고 있다.

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖ 주어진 그래프의 신장 트리를 찾으려면 사이클이 없도록 모든 점을 연결시키면 된다.

- 그래프의 점의 수가 n이면, 신장 트리에는 정확히 (n-1)개의 선분 있음.
- N-1 개 선분을 가진 트리에 선분을 하나 추가시키면, 반드시 사이클이 만들어짐



트리

점선으로 된 선분을 추가하여 만들어진 사이클

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖ 어떻게 최소 신장 트리를 찾을까?

- ➢ 무작위 기법
  - ✓ 가능한 신장 트리를 모두 구한 다음, 최소 비용의 신장 트리를 선택
  - ✓ 모든 정점 간에 간선이 있는 완전 그래프의 경우 신장 트리의 수는 n^{n-2}
    - 정점이 10 개: 1억 개의 신장 트리
    - 정점이 19 개: 약 5.4 ×1021 개의 신장 트리

❖ 욕심쟁이 방법 - 그리디 알고리즘

- ➢ 크러스컬 (Kruskal)과 프림 (Prim) 알고리즘 있음
- ➢ 알고리즘의 입력은 1개의 연결요소 (connected component)로 된 가중치 그래프이다.

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

- 가중치가 가장 작은 선분이 사이클을 만들지 않을 때에만 '욕심내어' 그 선분을 추가시킨다.

알고리즘 개요

1. 각 단계에서 가중치가 최소인 간선을 선택
   - 방법1) 최소값 찾기 알고리즘을 여러 번 실행 (x)
   - 방법2) **미리 모든 간선을 가중치에 따라 정렬**

2. 선택한 간선 때문에 사이클이 만들어진다면 선택한 간선을 버림
   - 방법1) 깊이 우선 탐색(DFS)이나 너비 우선 탐색(BFS)을 이용 (x)
   - 방법2) **배타적 집합(disjoint set)의 개념을 이용하는 union-find 연산**

3. 받아들여진 간선의 수가 n-1 개가 되면 종료
   - n개의 트리로 구성된 포리스트에서 출발하여 최종 1개의 트리를 생성

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

KruskalMST(G)

❖입력: 가중치 그래프 G=(V,E), |V|=n , |E|=m

❖출력: 최소 신장 트리 T

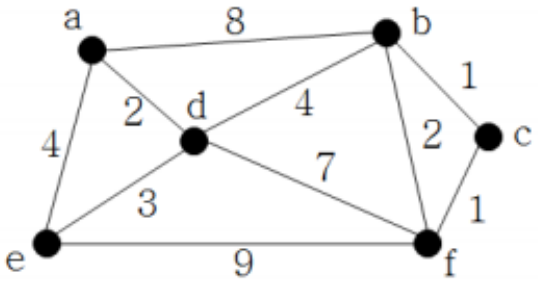
1. 가중치의 오름차순으로 선분들을 정렬한다. 정렬된 선분 리스트를 L이라고 하자.
2. $T = \emptyset$ // 트리 T를 초기화시킨다.
3. while (T의 선분 수 < n-1) {
4. L에서 가장 작은 가중치를 가진 선분 e를 가져오고, e를 L에서 제거한다.
5. if (선분 e가 T에 추가되어 사이클을 만들지 않으면) // feasibility check
6. e를 T에 추가시킨다.
7. else // e가 T에 추가되어 사이클이 만들어지는 경우
8. e를 버린다.
   }
9. return 트리 T // T는 최소 신장 트리이다.

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

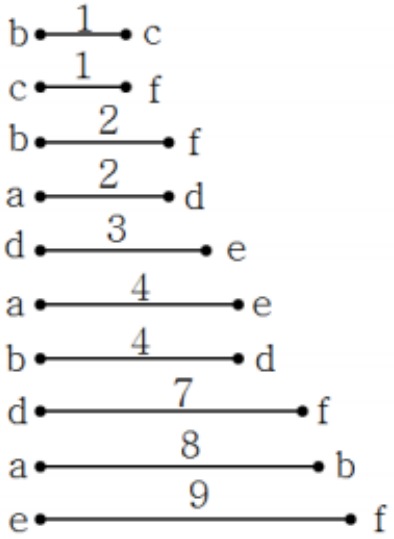
Cryptography & Information Security Laboratory

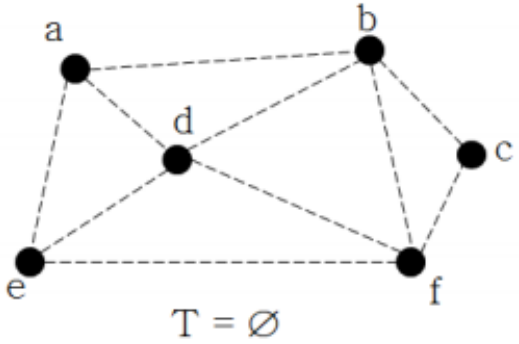
❖크러스컬 알고리즘

- 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.



정렬된 리스트 L

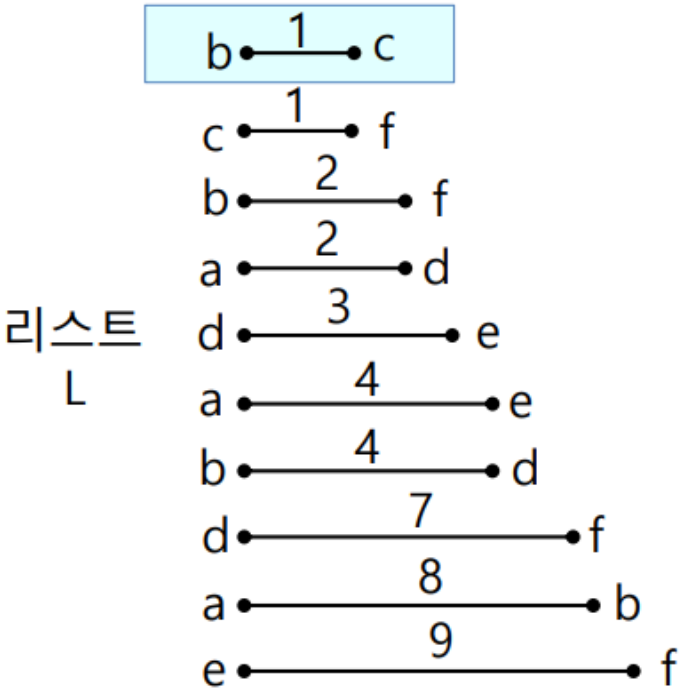


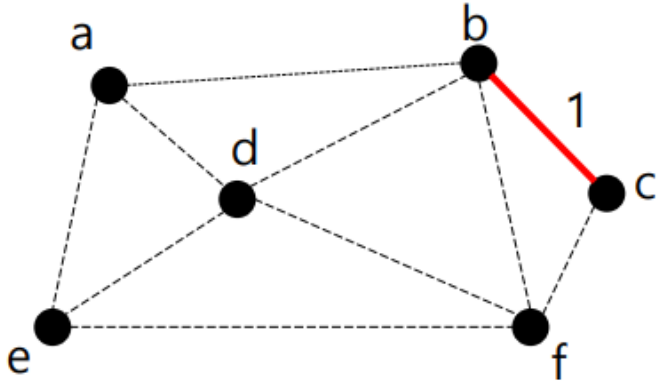


4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.





선분 (b,c) 추가

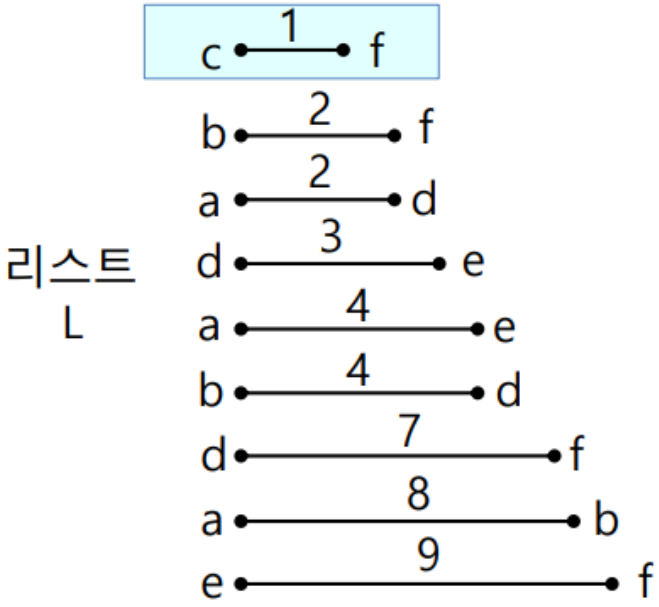
{a}, {b}, {c}, {d}, {e}, {f}

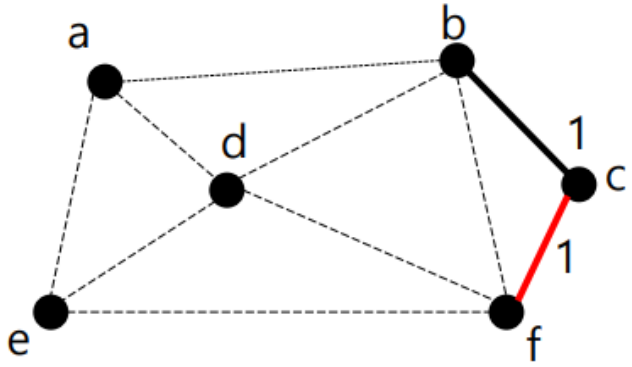
➔{a}, {b, c}, {d}, {e}, {f}

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.





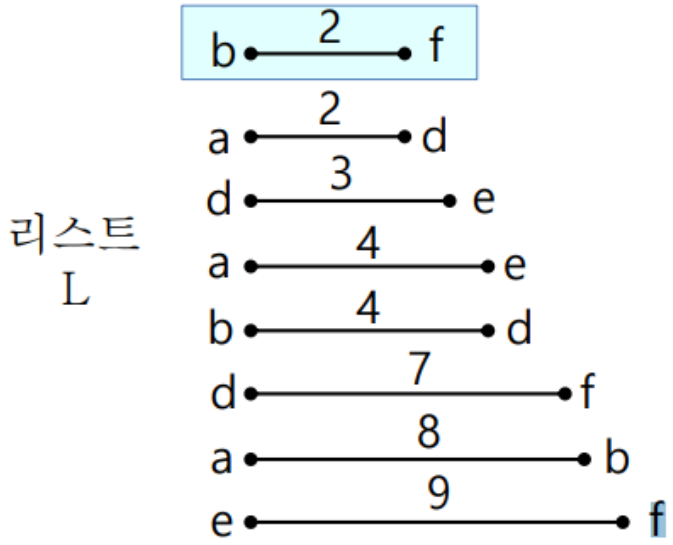
선분 (c,f) 추가

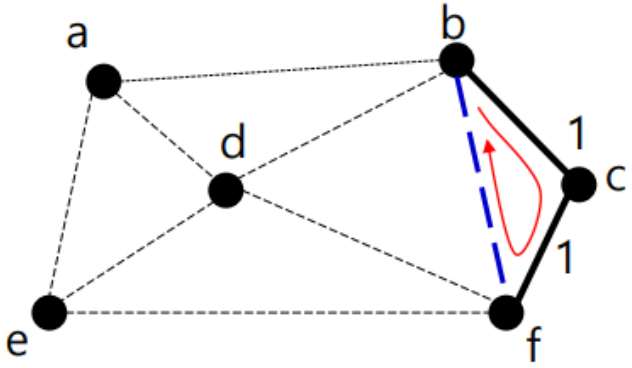
➔ {a}, {b, c, f}, {d}, {e}

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.





사이클 b-c-f-b

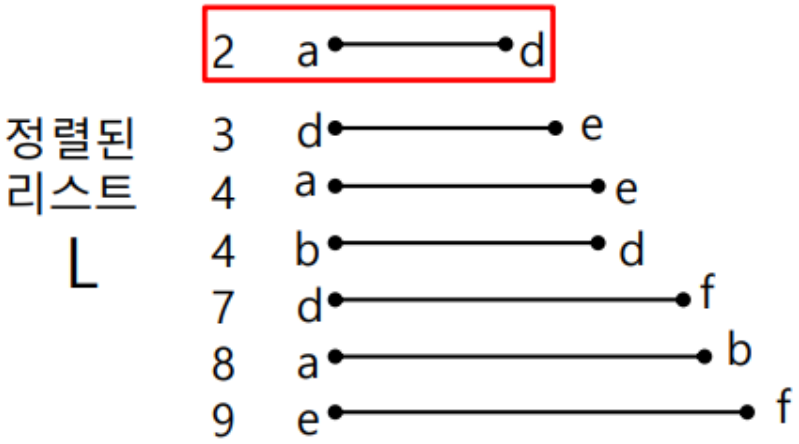
선분 (b,f) 버림

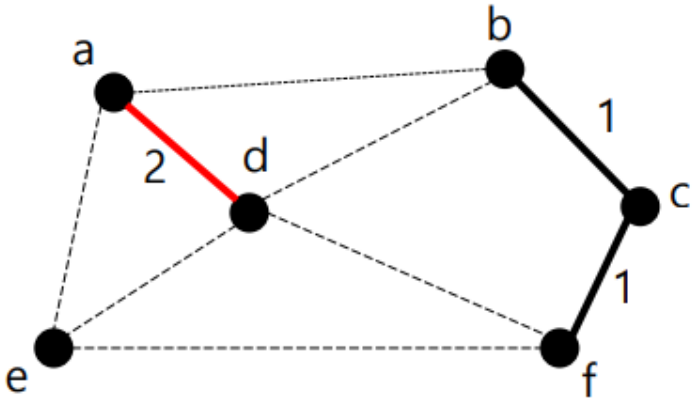
➔ {a}, {b, c, f}, {d}, {e}

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.





선분 (a,d) 추가

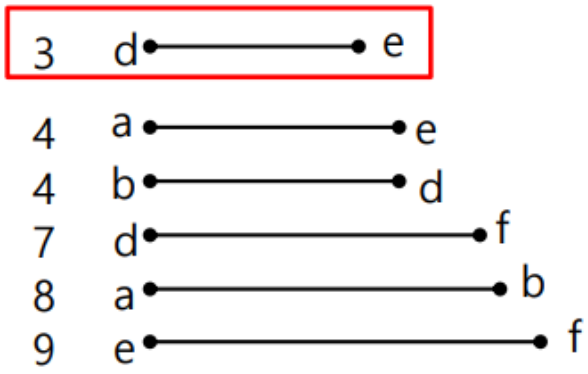
➔{a,d}, {b, c, f}, {e}

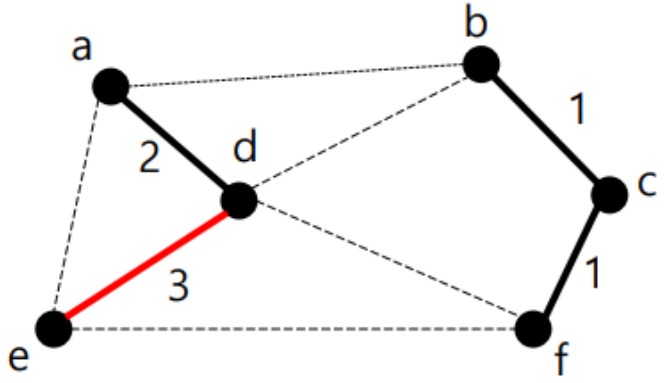
4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.

정렬된 리스트 L





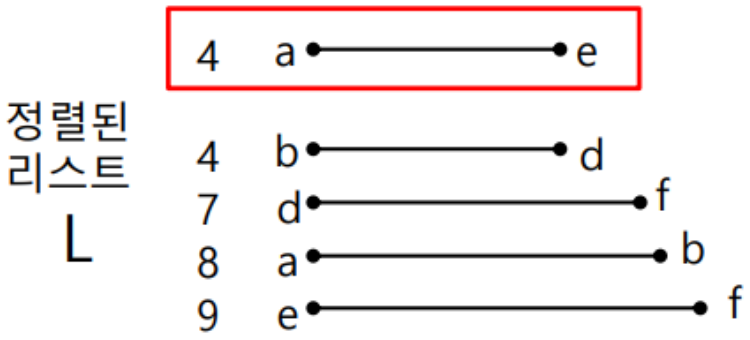
선분 (d,e) 추가

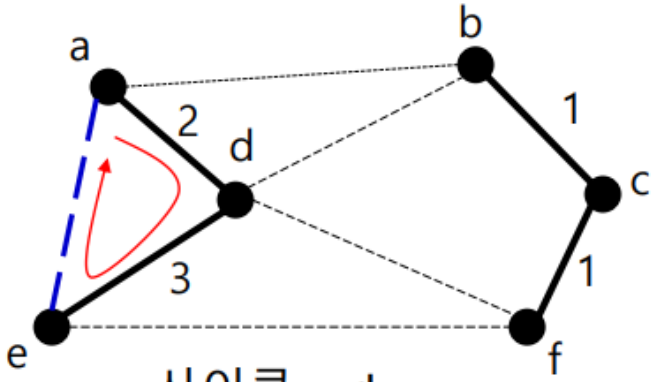
➔ {a,d,e}, {b, c, f}

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.



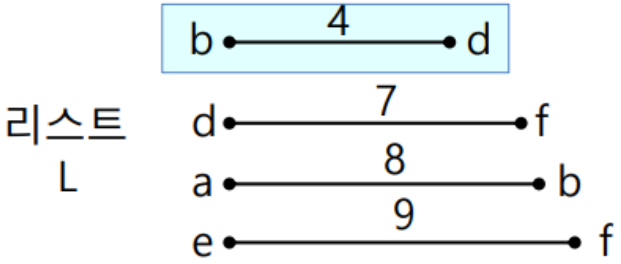


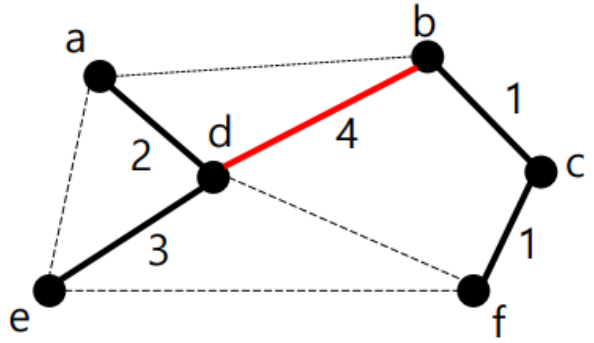
➔ {a,d,e}, {b, c, f}

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

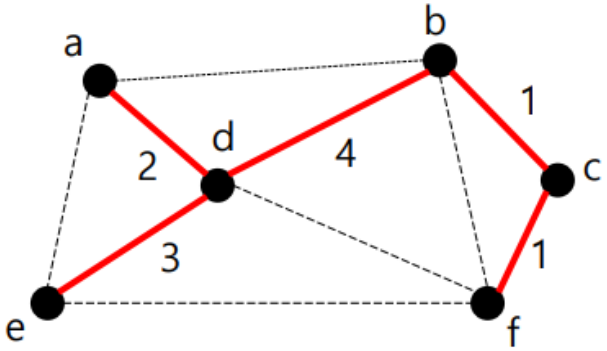
❖크러스컬 알고리즘

➢ 다음의 그래프에서 KruskalMST 알고리즘이 최소 신장 트리를 찾는 과정을 살펴보자.





선분 (b,d) 추가



<최종 해>

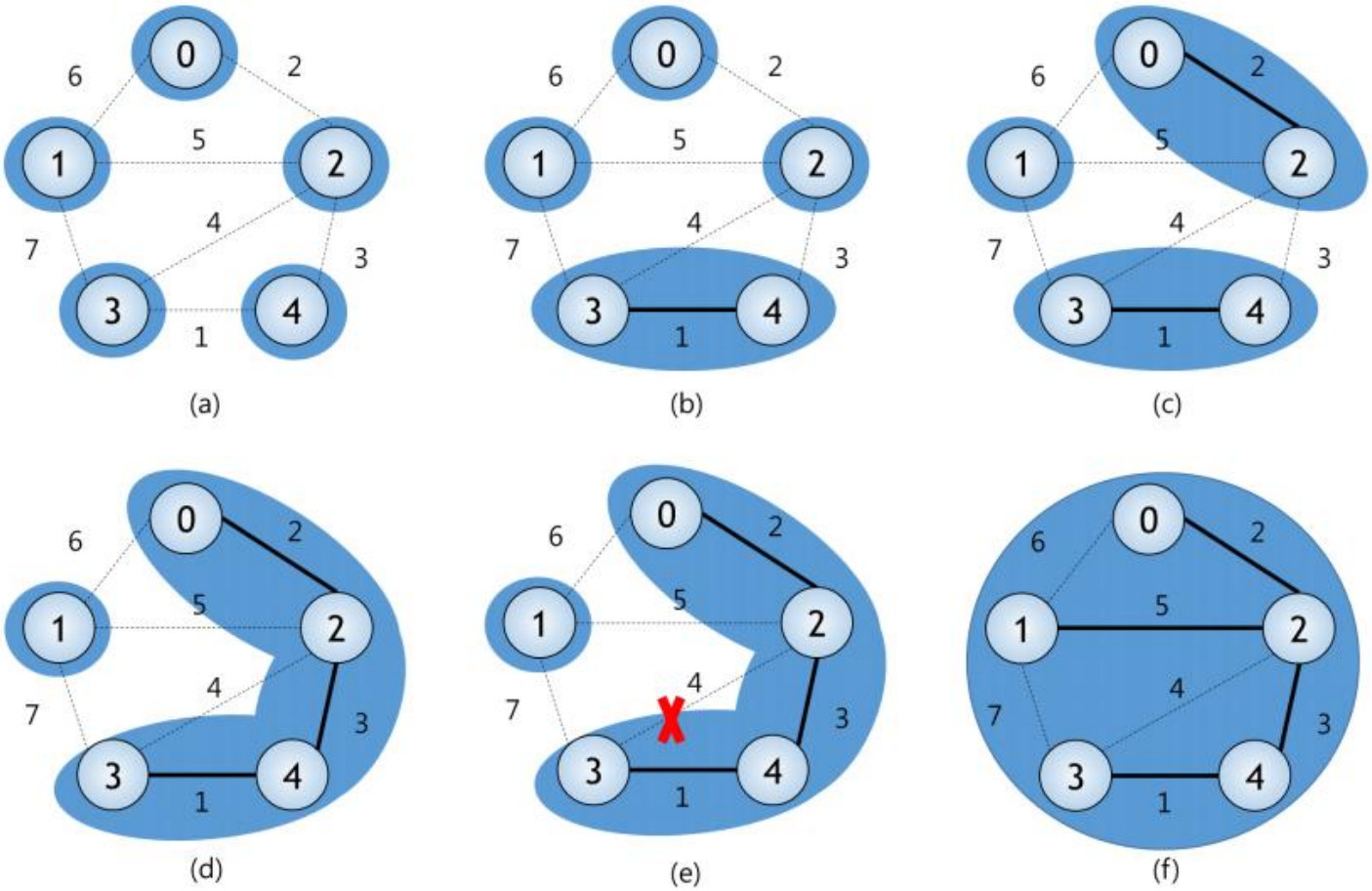
➔ {a,d,e, b, c, f}

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

- ➢ 예제)



4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

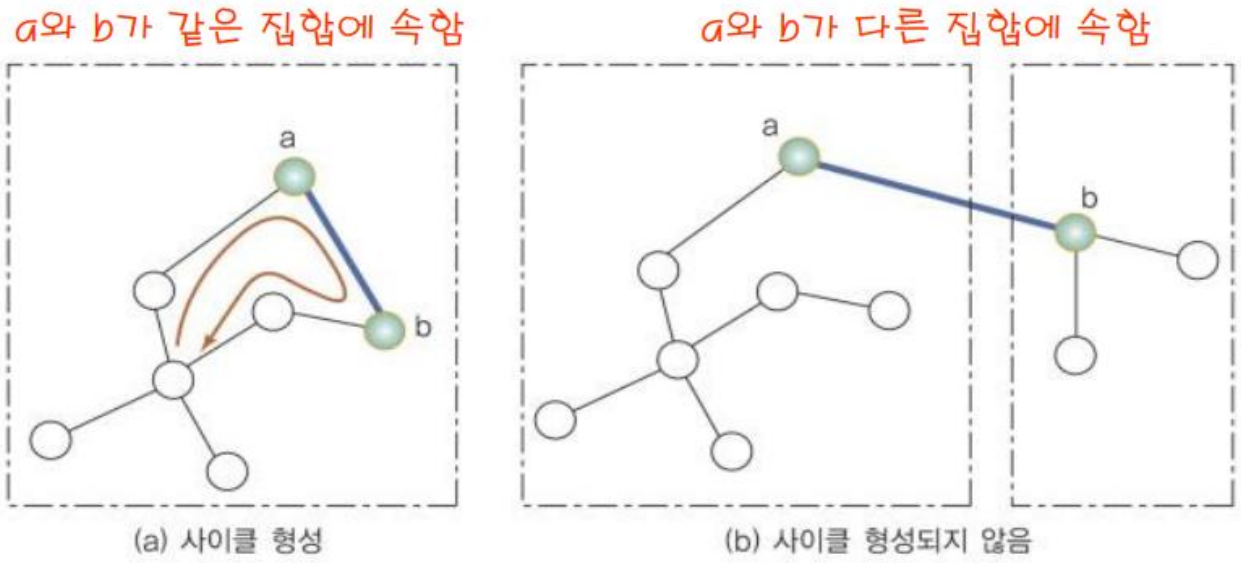
➢ How to check cycle?

```
edge_set kruskal_MST(edge_set E, int n) {
  sort(E);
  edge_set MST_E = { };
  for (i=0; i<n; i++) init_set(i); // n개의 집합(트리)을 생성
  while (MST_E의 간선 수 < n-1) {
  e=(u, v) = E의 최소 가중치 간선;
    E = E - {(u, v)} ;
    if (find(u) ≠ find(v)) // u와 v가 다른 집합(트리) 원소
    {
      MST_E = MST_E ∪ {(u, v)} ;
      union(u, v); // 두 집합(트리)을 합병
    }
  }
  return MST_E;
}
```

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖크러스컬 알고리즘

- How to check cycle?
  - Cycle 발생 유무 체크는 어떻게?



(a) 사이클 형성 (b) 사이클 형성되지 않음

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

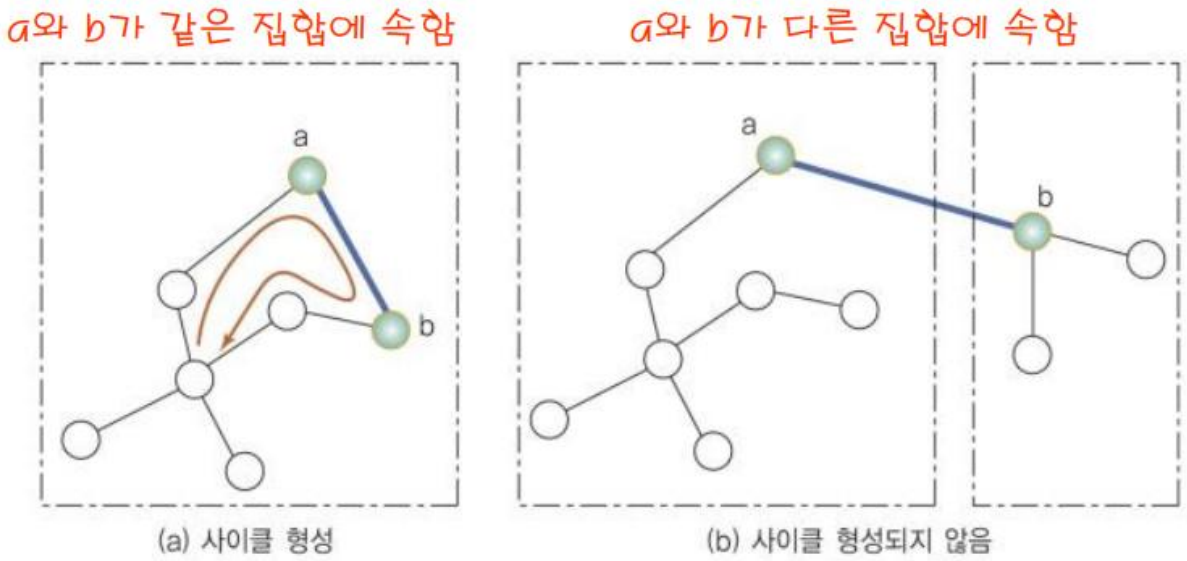
Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

- How to check cycle?
  - Cycle 발생 유무 체크는 어떻게?

❖ union-find 알고리즘

- (두 원소가 다른 집합에 속할 때)두 집합들의 합집합을 생성하는 union
- 특정 원소가 어떤 집합에 속하는지 알아내는 find
- Kruskal의 MST 알고리즘에서 사이클 검사에 사용



4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

- How to check cycle?
  - Cycle 발생 유무 체크는 어떻게?

❖ union-find 알고리즘

- 배열을 사용하여 각 집합을 tree 형태로 구성 (자료구조 !!)
- Find (a) : a 가 포함된 집합을 구성하는 tree 의 root 를 return
- Union(a, b) : a와 b가 서로 다른 집합에 속하고, 각각 속한 tree의 root라고 할 때, 두 tree를 merge하여 대응되는 두 집합의 합집합 (union)을 표현하는 tree를 구성한다.
- 두 원소 v와 u 및 그들의 속한 집합들을 union하려면,

```
a = find( v ); b = find ( u );      // find the roots of v and u.
if ( a != b ) {     // if the same root, v and u are
                    // in the same subset
union ( a, b );
}
```

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

➢ 시간복잡도

KruskalMST(G)

- 입력: 가중치 그래프 $G = (V, E), |V| = n, |E| = m$
- 출력: 최소 신장 트리 T

1. 정렬된 선분 리스트 L
2. $T = \emptyset$
3. *while* (T의 선분 수 $< n - 1$) {
4. $get \min(e)\ in\ L,\ \ remove(e)\ from\ L$
5. if (선분 e가 T에 추가되어 사이클을 만들지 않으면)
6. $add(e)\ to\ L$
7. $else$
8. $drop(e)$
   }
9. $return\ T$

Line 1: 정렬하는데 $O(m \log m)$ 시간. 단, m은 입력 그래프에 있는 선분의 수

Line 2: T를 초기화하는 것이므로 $O(1)$ 시간

Line 3~8의 while-루프[m x $O(log^* m)$]

최악의 경우 m번 수행. 즉, 그래프의 모든 선 분이 while-루프 내에서 처리되는 경우.

또한, while-루프 내에서 L로부터 가져온 선분 e가 사이클을 만드는지를 검사하는 데, 이는 $O(log^* m)$ 시간이 걸린다.

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘

- 따라서 크러스컬 알고리즘의 시간복잡도는 $O(m \log m) + O(m log^* m) = \mathbf{O(m \log m)}$
  - $O(m \log^* m) \fallingdotseq O(m)$
  - $\log^* n$: log 함수를 n에 연속해서 몇 번이나 적용해야 1이하가 되는가

$$\log^* n := \begin{cases} 0 & \text{if } n \leq 1; \\ 1 + \log^*(\log n) & \text{if } n > 1 \end{cases}$$

x	$\lg^* x$
$(-\infty, 1]$	0
(1, 2]	1
(2, 4]	2
(4, 16]	3
(16, 65536]	4
(65536, 2^{65536}]	5

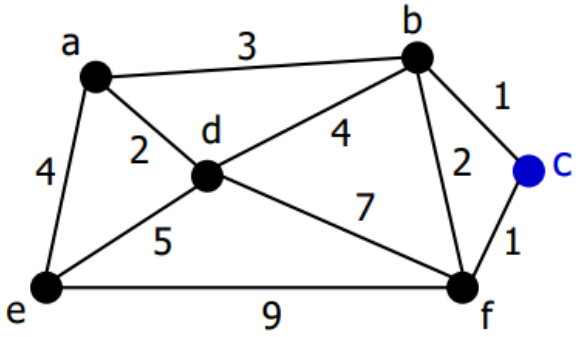
  - 예) 2^{65536}은 매우 큰 수이지만 $\log \log \log \log \log 2^{65536} = 1$이므로, $\log^* 2^{65536} = 5$

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖프림 알고리즘

- 주어진 가중치 그래프에서 (1) 임의의 점 하나를 선택한 후, (2) 그 점으로 부터 각 점의 거리를 계산 후, (3) 가장 짧은 거리를 갖는 점을 추가시켜 트리를 만든다. 결국, (n-1) 개의 선분을 하나씩 추가시켜 트리를 만든다는 효과와 같다.

  - 즉, 추가되는 선분은
    - 현재까지 만들어진 트리에 연결시킬 때
    - '욕심을 내어서' 항상 최소의 가중치로 연결되는 선분이다.



4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖프림 알고리즘

- 입력: 가중치 그래프 $G = (V, E), |V| = n, |E| = m$
- 출력: 최소 신장 트리 T

PrimMST(G)

1. 그래프 G에서 임의의 점 p를 시작점으로 선택하고, $D[p] = 0$으로 놓는다.
   // 배열 D[v]는 T에 있는 점 u와 v를 연결하는 선분의 최소 가중치를 저장하기 위한 원소이다.
2. for (점 p가 아닌 각 점 v에 대하여) { // 배열 D의 초기화
3. if (선분 (p, v)가 그래프에 있으면)
4. $D[v]$ = 선분 (p, v)의 가중치
5. $else$
6. $D[v] = \infty$
   }

$V = \{ a\ b\ c\ d\ e\ f \}$

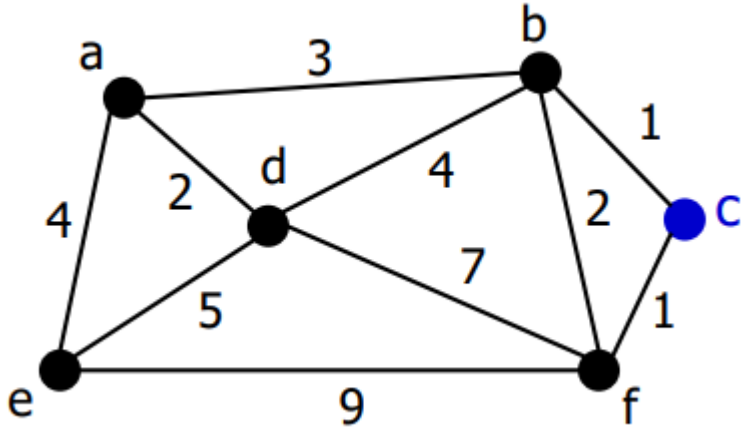
$D[v] = [\ \infty\ \ 1\ \ 0\ \infty\ \infty\ \ 1\ ]$

7. $T = \{p\}$ // 초기에 트리 T는 점 p만을 가진다.
8. $while$ (T에 있는 점의 수 $<\ n$) {
9. T에 속하지 않은 각 점 v에 대하여, $D[v]$가 최소인 점 v_{min}과 연결된 선분 (u, v_{min})을 T에 추가한다.
   단, u 는 T에 속한 점이고, 점 v_{min}도 T에 추가된다.
10. for (T에 속하지 않은 각 점 w에 대해서) {
11. if (선분 $(v_{min}$,W)의 가중치 $<\ D[w]$)
12. $D[w]$ = 선분 (v_{min}, w)의 가중치 // D[w]를 갱신
    }
    }
13. $return\ T$ // T는 최소 신장 트리이다

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖프림 알고리즘

- **PrimMST(G)**알고리즘 수행 과정



- Line 1: 임의의 시작점으로 점 c가 선택되었다고 가정한다. $D[c] = 0$으로 초기화 시킨다.

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖프림 알고리즘

- Line 2~6: 배열 D의 초기화
- 시작점 c와 선분으로 연결된 각 점 v에 대해서, $D[v]$를 각 선분의 가중치로 초기화
- 나머지 각 점 w에 대해서, $D[w]$는 ∞로 초기화
- Line 7: 초기에는 트리 T는 점 c만 포함, 즉, $T = \{c\}$로 초기화.

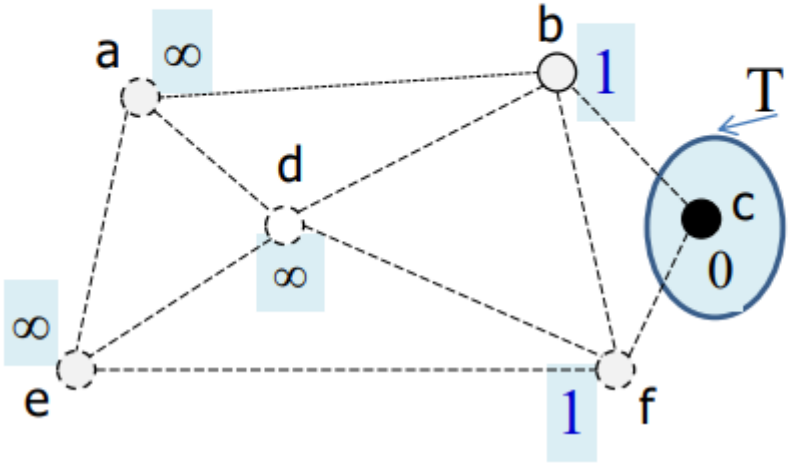
D[v]∈{D[b]=1, D[f]=1}

D[w]∈{D[a], D[d], D[e]}

$D[]$ = a b c d e f

[∞ 1 0 ∞ ∞ 1]

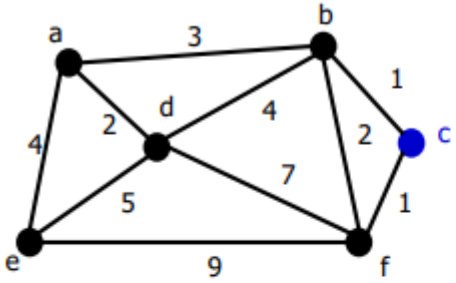
실제로는 D에 edge index를 저장

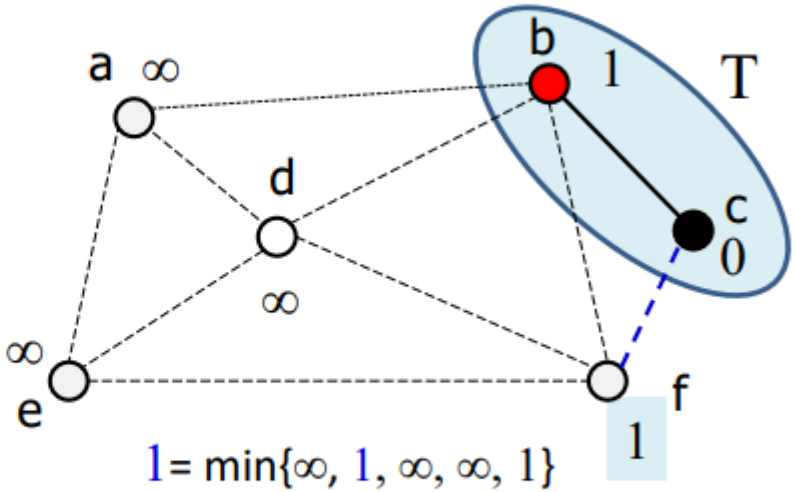


4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖프림 알고리즘

- Line 8의 while-루프의 조건이 '참'이다. 즉, 현재 트리 T에는 점 c만이 있다. 따라서 line 9에서 T에 속하지 않은 각 점 v에 대하여, $D[v]$가 최소인 점 v_{min}을 선택한다.
- $D[b] = D[f] = 1$로서 최소값이므로 점 b나 점 f 중 택일. 여기서는 점 b를 선택한다.
- 따라서 점 b와 선분 (c, b)가 트리 T에 추가된다.



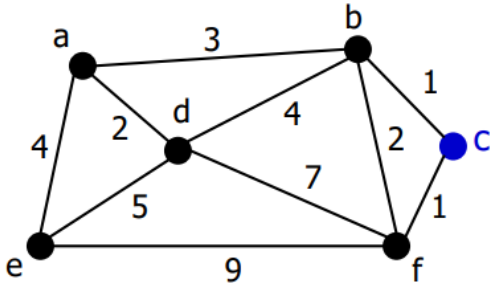


$D[v_{min}] = D[b]$ ➔ $v_{min} = b$

$D[]$ =	a	b	c	d	e	f
	∞	1	0	∞	∞	1

❖프림 알고리즘

- Line 10~12: 점 b에 연결된 점 a와 d의 $D[a]$와 $D[b]$를 각각 3과 4로 갱신한다.
  - 점 f는 점 b와 선분으로 연결은 되어있으나, 선분 (b, f)의 가중치인 2가 현재 $D[f]$보다 크므로 $D[f]$를 갱신 안됨

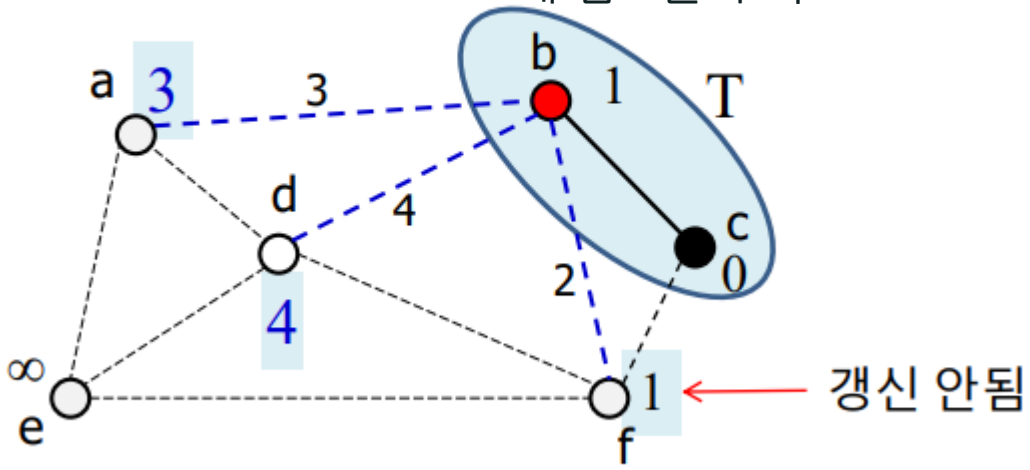


$D[]$ = a b c d e f
[∞ 1 0 ∞ ∞ 1]

$D[]$ = a b c d e f
[**3** **1** 0 **4** ∞ 1]

D[w]∈{D[a]=3, D[d]=4, D[e]=∞, D[f]=1}

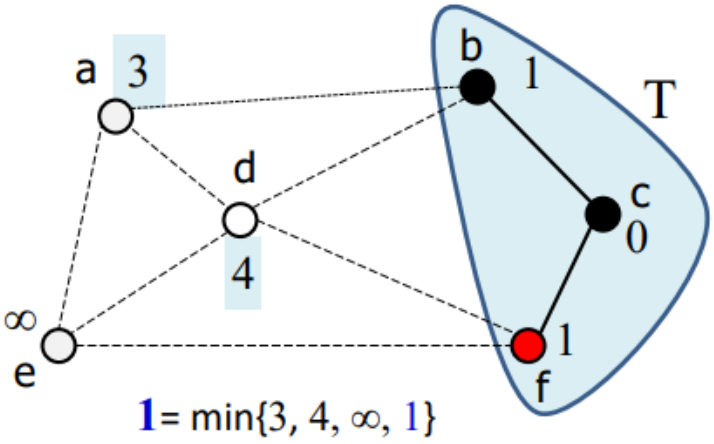
T에 점 b를 추가



4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

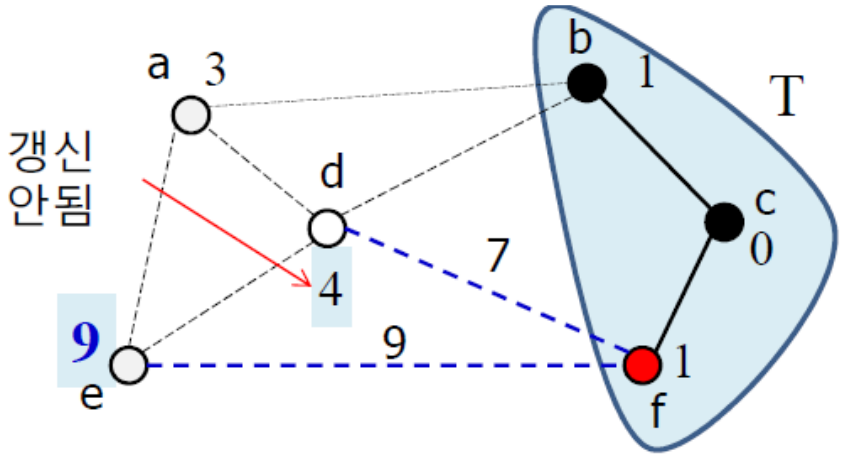
❖프림 알고리즘

- Line 8의 while-루프의 조건이 '참'이므로, line 9에서 T에 속하지 않은 각 점 v에 대하여, v_{min}인 점 f를 찾고, 점 f와 선분 (c, f)를 T에 추가시킨다.



1= min{3, 4, ∞, 1}

T에 점 f을 추가



$D[]$ = a b c d e f
[∞ 1 0 ∞ ∞ 1]

$D[]$ = a b c d e f
[**3** **1** 0 **4** ∞ 1]

$D[]$ = a b c d e f
[3 1 0 4 **9** 1]

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖프림 알고리즘

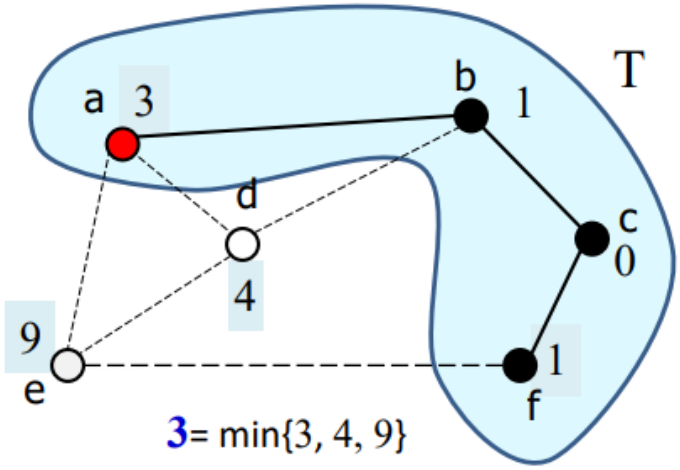
- 그 다음부터는 점 a와 선분 (b, a), 점 d와 선분 (a, d)가 차례로 T에 추가되고, 최종적으로 점 e와 선분 (a, e)가 추가되면서, 최소 신장 트리 T가 완성된다.
- Line 13에서는 T를 리턴하고, 알고리즘을 마친다. 다음의 그림들이 위의 과정을 차례로 보여준다.

$D[]$ = a b c d e f
[∞ 1 0 ∞ ∞ 1]

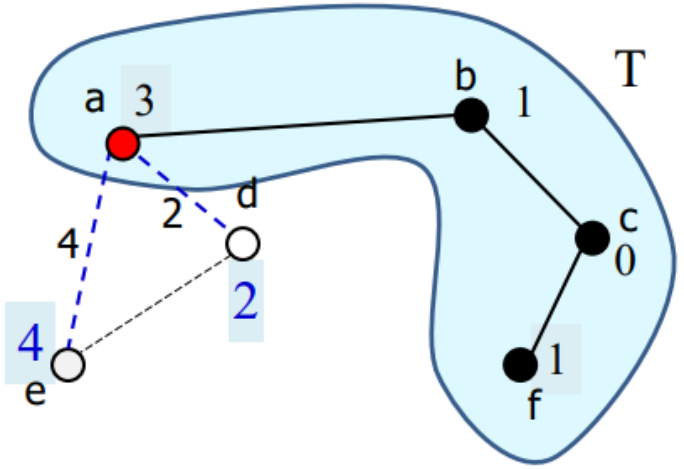
$D[]$ = a b c d e f
[**3** **1** 0 **4** ∞ 1]

$D[]$ = a b c d e f
[3 1 0 4 **9** 1]

$D[]$ = a b c d e f
[3 1 0 **2** **4** 1]

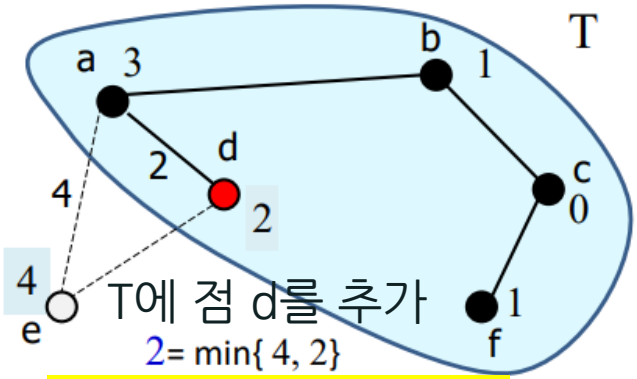


T에 점 a을 추가

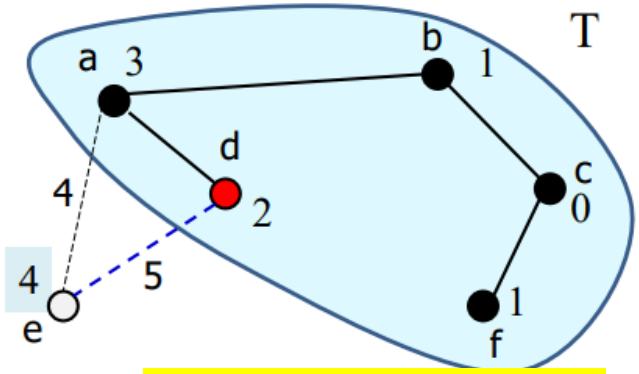


4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

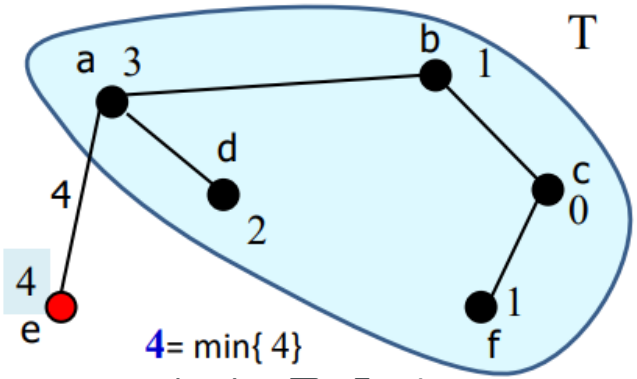
❖프림 알고리즘

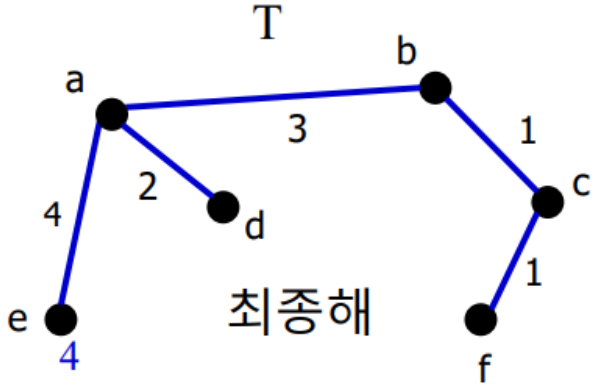


$D[]$ = a b c d e f
[3 1 0 2 4 1]



$D[]$ = a b c d e f
[3 1 0 2 4 1]

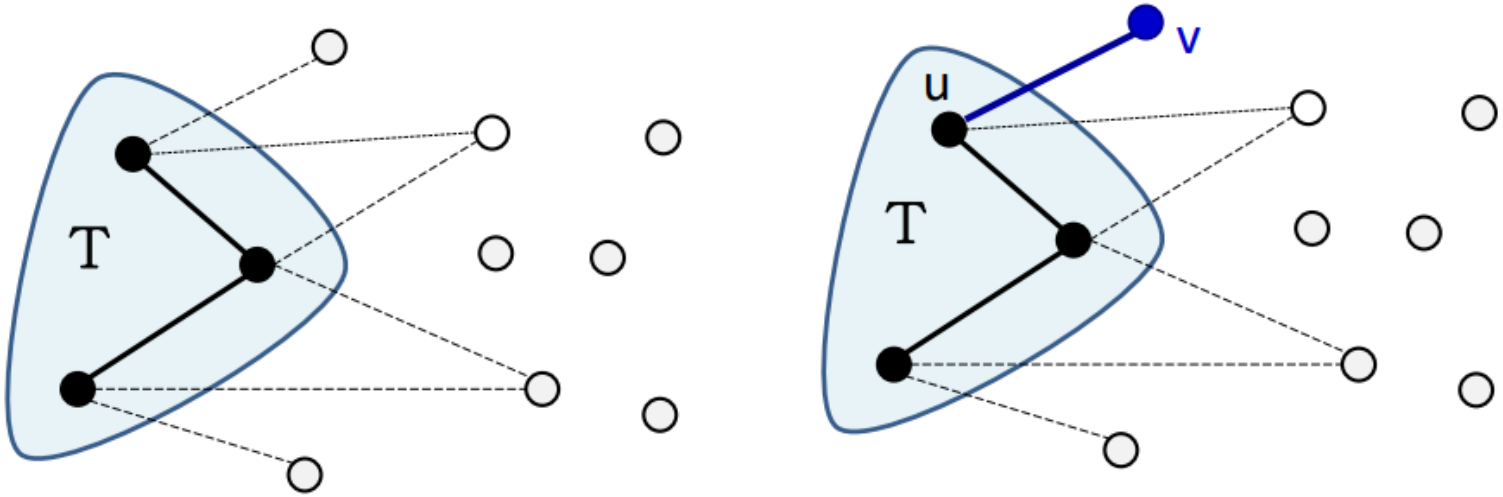




4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

❖프림 알고리즘

- PrimMST 알고리즘이 **최종적으로 리턴하는 T에는 왜 사이클이 없을까**?
  - 프림 알고리즘은 항상 T 밖에 있는 점을 추가하므로 사이클이 형성되지 않는다.



4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖프림 알고리즘

➢ 시간복잡도

- while-루프가 $(n-1)$번 반복되고, 1회 반복될 때 line 9에서 T에 속하지 않은 각 점 v 에 대하여, $D[v]$가 최소인 점 v_{min}을 찾는데 $O(n)$ 시간이 걸린다.
  - 1차원 배열 D에서 (현재 T에 속하지 않은 점들에 대해서) 최소값을 찾는 것이고, 배열의 크기는 그래프의 점 의 수인 n이기 때문

- 프림 알고리즘의 시간복잡도: $(n-1) \times O(n) = \mathbf{O(n^2)}$

8. *while* (T에 있는 점의 수 $< n$) { // $(n-1)$ 번 반복 * $O(n)$
9. T에 속하지 않은 각 점 v에 대하여, $D[v]$가 최소인 점 v_{min}과 연결된 선분 (u, v_{min})을 T에 추가한다.
   단, u 는 T에 속한 점이고, 점 v_{min}도 T에 추가된다.
10. *for* (T에 속하지 않은 각 점 w에 대해서) {// $O(n)$
11. if (선분 $(v_{min}$,w)의 가중치 $<$ $D[w]$)
12. $D[w]$ = 선분 (v_{min}, w)의 가중치 // D[w]를 갱신
    }
    }
13. *return* T // T는 최소 신장 트리이다

4.2 최소 신장 트리 (Minimum Spanning Tree, MST)

Cryptography & Information Security Laboratory

❖크러스컬 알고리즘 VS 프림 알고리즘

- **크러스컬 알고리즘**에서는 선분이 1개씩 T에 추가되는데, 이는 마치 n개의 점들이 각각의 트리인 상태에서 선분이 추가되면 2개의 트리가 1개의 트리로 합쳐지는 것과 같다.
- 크러스컬 알고리즘은 이를 반복하여 1개의 트리인 T를 만든다. 즉, n개의 트리들이 점차 합쳐져서 1개의 신장 트리가 만들어진다.

- **프림 알고리즘**에서는 T가 점 1개인 트리에서 시작되어 선분을 1개씩 추가시킨다. 즉, 1개의 트리가 자라나서 신장 트리가 된다.

- 최소 신장 트리의 응용
  - 최소 비용으로 선로 또는 파이프 네트워크 (인터넷 광 케이블 선로, 케이블 TV선로, 전화선로, 송유관로, 가스관로, 배수로 등)를 설치하는데 활용
  - 여행자 문제 (Traveling Salesman Problem)를 근사적으로 해결하는데 이용

4.3 최단 경로 찾기 (Shortest Path)

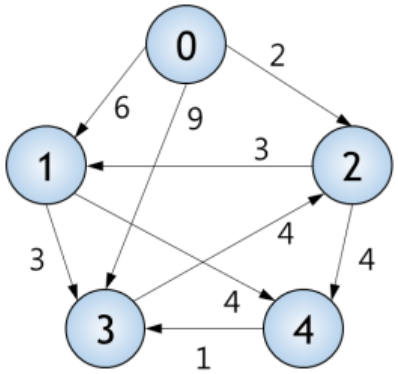
❖최단 경로 (Shortest Path) 문제는 주어진 가중치 그래프에서 어느 한 출발점에서 또 다른 도착점까지의 최단 경로를 찾는 문제이다.

❖최단 경로를 찾는 가장 대표적인 알고리즘은

- 방법1) 무작위 기법
  - 모든 도시들이 다 연결 되어 있다면 출발점에서 가능한 경로의 수는 Θ(n!)



- 방법2) 다익스트라 (Dijkstra) 최단 경로 알고리즘이며,
  - 그리디 알고리즘.
  - (결과적으로) 출발점에서 가까운 도시부터 차례대로 최단 경로를 탐색
  - 한번 결정된 최단 경로는 절대 바뀌지 않으며 출발점에서 더 먼 도시의 최단 경로를 구할 때 이용



최단경로	길이
0 → 2	2
0 → 2 → 1	5
0 → 2 → 4	6
0 → 2 → 4 → 3	7

4.3 최단 경로 찾기 (Shortest Path)

❖최단경로 찾기 알고리즘의 종류

- 단일 시작점(single source) 최단 경로 알고리즘: 출발지점을 하나 선택하면 나머지 다른 모든 지점까지의 최단 경로를 알려주는 유형
- 모든 쌍(all-pairs) 최단 경로 알고리즘: 모든 지점들 간의 최단 경로를 알려주는 유형

❖최단경로 찾기 알고리즘: **프림의 최소 신장 트리 알고리즘과 거의 흡사한 과정으로 진행**

❖프림알고리즘과의 차이점:

1. 프림 알고리즘은 임의의 점에서 시작하나**, 다익스트라 알고리즘은 주어진 출발점에서 시작**

2. 프림 알고리즘은 트리에 하나의 점 (선분)을 추가시킬 때 **현재 상태의 트리**에서 가장 가까운 점을 추가.
   그러나 **다익스트라의 알고리즘은 출발점으로부터 최단 거리가 확정되지 않은 점들 중에서 출발점으로부터 가장 가까운 점을 추가**하고, 그 점의 최단 거리를 확정.

4.3 최단 경로 찾기 (Shortest Path)

❖MST를 구하는 프림 알고리즘과 원리가 거의 동일

❖처음부터 끝까지 트리를 유지

- 출발점을 시작 트리로 초기화하고 매 단계에서 하나의 노드와 간선을 붙여나감

❖노드를 추가 한다 = 그 지점까지의 최단 경로를 한다

- 처음에는 최단 경로가 확정된 노드 집합={출발 노드}
- 매 단계마다 하나씩 노드를 늘려가면서 모든 노드들을 포함할 때까지 반복

❖욕심쟁이 방법이니 만큼 노드를 선택하는 방법이 핵심

- **기준: 출발 도시로부터의 최단 경로가 가장 짧은 노드**
- **단, 이미 최단 경로가 밝혀진 노드들만 경유**

4.3 최단 경로 찾기 (Shortest Path)

Cryptography & Information Security Laboratory

❖다익스트라 알고리즘

ShortestPath(G, s)

❖ 입력: 가중치 그래프 $G = (V, E), |V| = n, |E| = m$

❖ 출력: 출발점 s로부터 $(n-1)$개의 점까지 각각 최단 거리를 저장한 배열 D

1. 배열 D를 ∞로 초기화시킨다. 단, $D[s] = 0$으로 초기화한다.
   // 배열 $D[v]$에는 출발점 s로부터 점 v까지의 거리가 저장된다.
   // 각 점 v에 대해서 $D[v] = \infty$로 초기화

2. *while* (s로부터의 최단 거리가 확정되지 않은 점이 있으면) {
3. 현재까지 s로부터 최단 거리가 확정되지 않은 각 점 v에 대해서 최소의 $D[v]$의 값을 가진 점 v_{min}을 선택하고, 출발점 s로부터 점 v_{min} 까지 의 최단 거리 $D[v_{min}]$을 확정
4. s로부터 현재보다 짧은 거리로 점 v_{min} 을 통해 우회 가능한 각 점 w에 대해서 $D[w]$를 갱신
   }

5. *return* D

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

ShortestPath(G, s)

❖ 입력: 가중치 그래프 $G = (V, E), |V| = n, |E| = m$

❖ 출력: 출발점 s로부터 $(n-1)$개의 점까지 각각 최단 거리를 저장한 배열 D

1. 배열 D를 ∞로 초기화시킨다. 단, $D[s] = 0$으로 초기화한다.
   // 배열 $D[v]$에는 출발점 s로부터 점 v까지의 거리가 저장된다.
   // 각 점 v에 대해서 $D[v] = \infty$로 초기화
2. *while* (s로부터의 최단 거리가 확정되지 않은 점이 있으면) {
3. 현재까지 s로부터 최단 거리가 확정되지 않은 각 점 v에 대해서 최소의 $D[v]$의 값을 가진 점 v_{min}을 선택하고, 출발점 s로부터 점 v_{min} 까지 의 최단 거리 $D[v_{min}]$을 확정
4. s로부터 현재보다 짧은 거리로 점 v_{min} 을 통해 우회 가능한 각 점 w에 대해서 $D[w]$를 갱신
   }
5. *return* D

Line 2~4의 while-루프는 (n-1)회 수행됨.

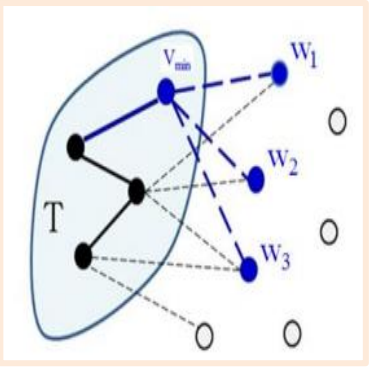
현재까지 s로부터 최단 거리가 확정된 점들의 집합을 T라 하면, V-T는 현재까지 s로부터 최단 거리가 확정되지 않은 점들의 집합.

따라서 V-T에 속한 각 점 v에 대해서 D[v]가 최소인 점 v_{min} 을 선택하고, v_{min}의 최단 거리를 확정시킨다.

즉, $D[v_{min}] \leq D[v], v \in V - T$이다.
'확정한다는 것'은 2가지의 의미를 갖는다.

1. $D[v_{min}]$이 확정된 후에는 다시 변하지 않는다.
2. 점 v_{min} 이 T에 포함된다

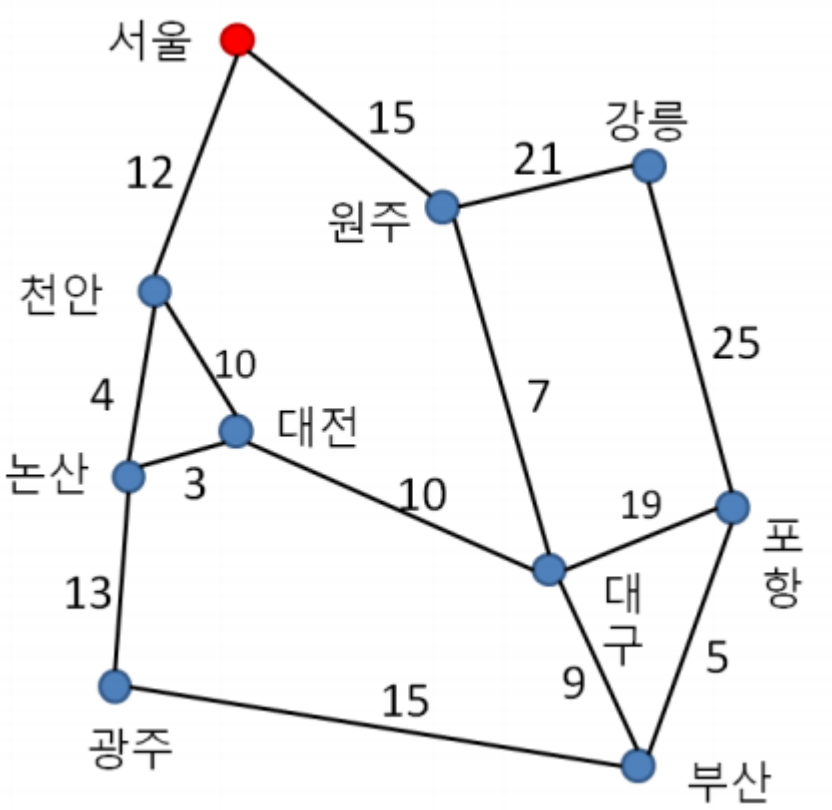
다음 그림은 v_{min} 이 T에 포함된 상태를 보이고 있는데, v_{min} 에 인접한 점 w_1, w_2, w_3 각각에 대해서 만일 **$(D[vmin]$ +선분 (v, w_i)의 가중치$) < D[w_i]$**이면, $D[w_i] = (D[v_{min}]$ +선분(v_{min}, w_i)의 가중치)로 갱신하는 것을 나타낸다



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

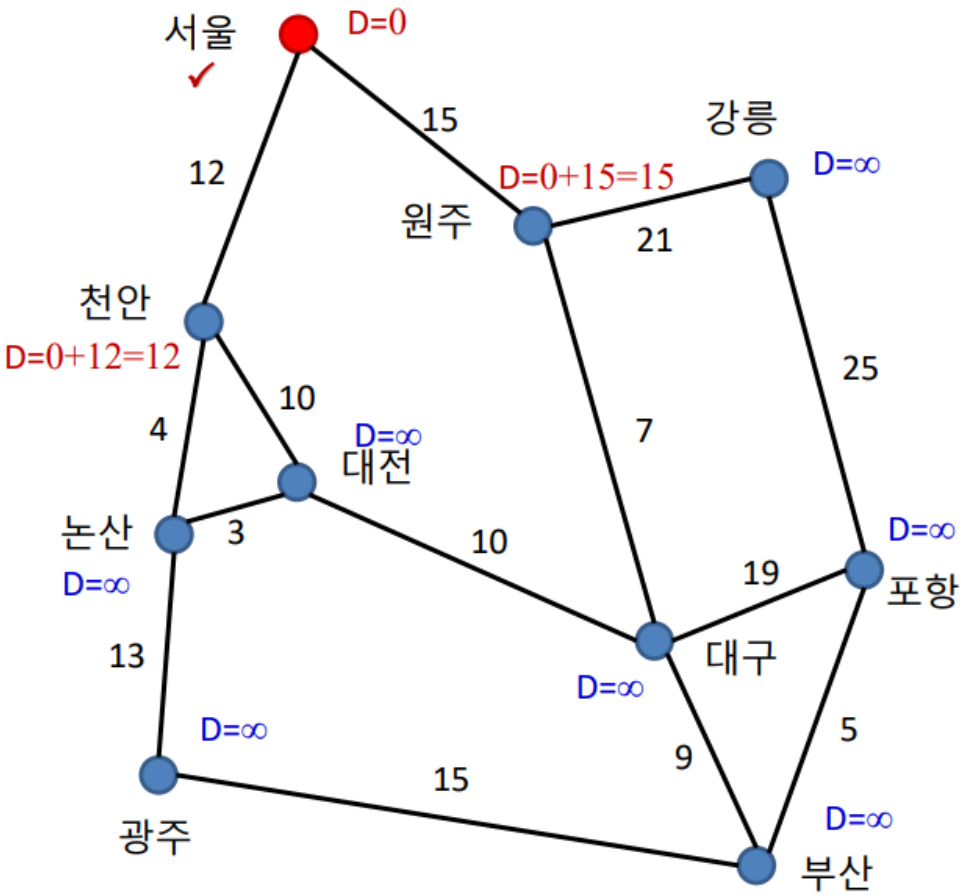
- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



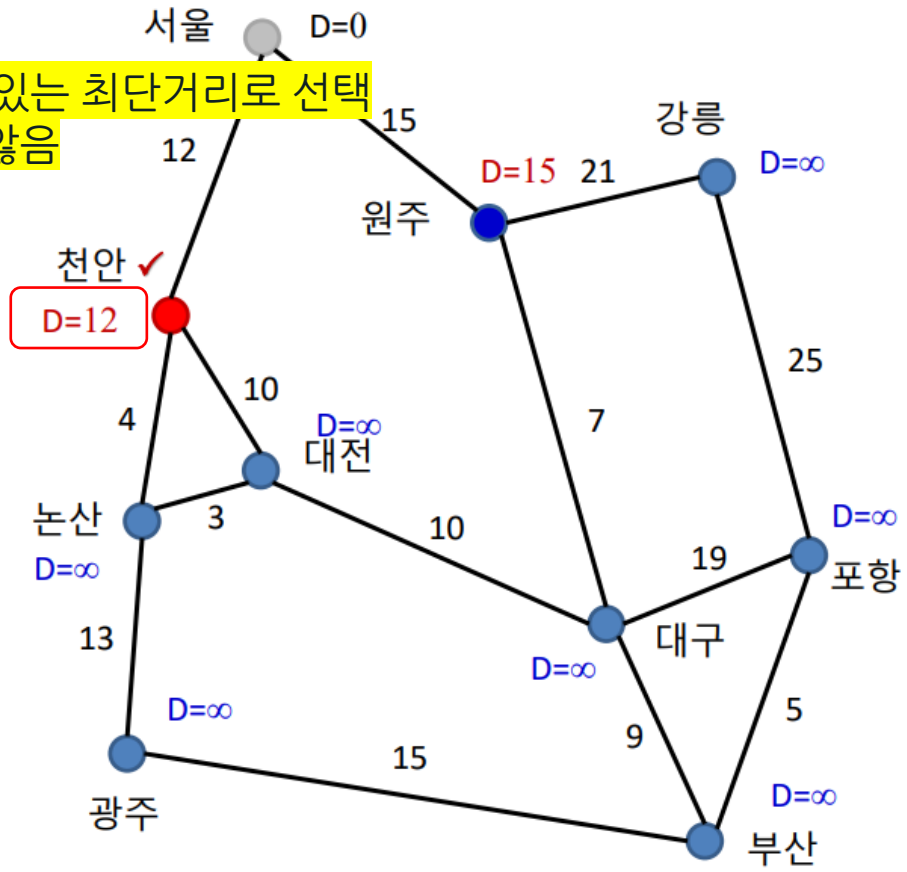
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	∞	15	12	∞	∞	∞	∞	∞	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)

서울에서 천안으로 갈 수 있는 최단거리로 선택되었으므로 값은 바뀌지 않음



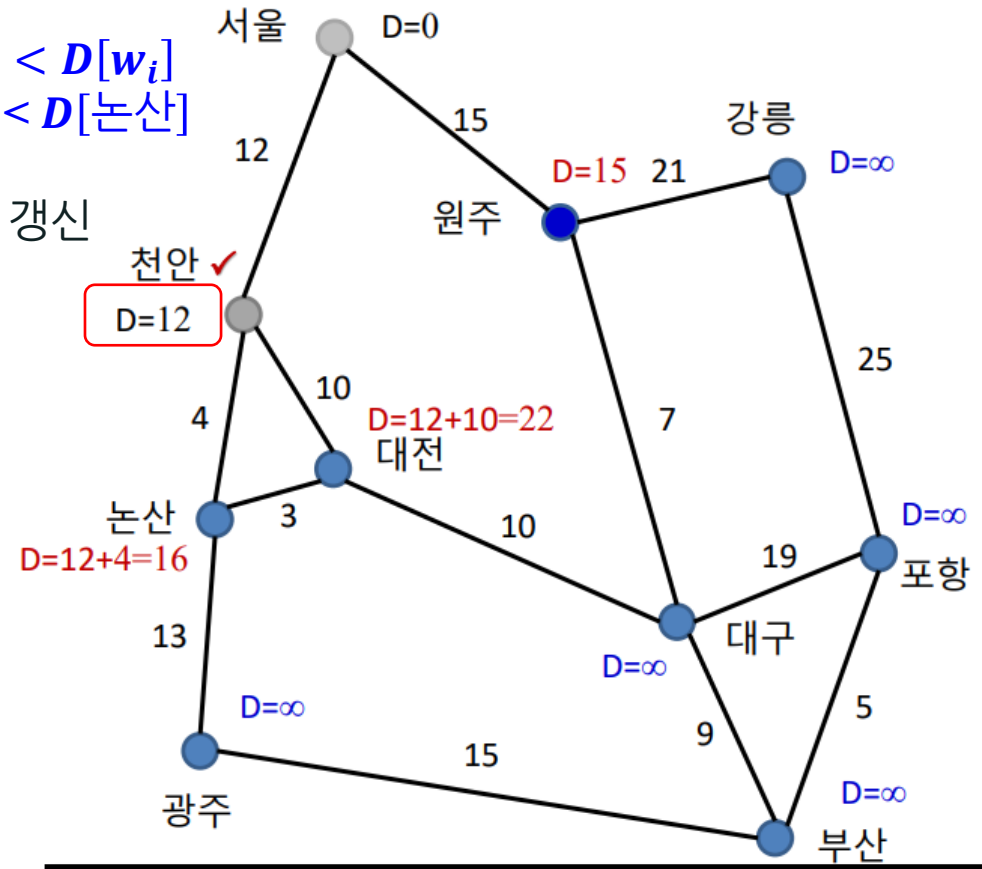
	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	∞	15	12	16	22	∞	∞	∞	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)

$D[vmin]$ +선분 (v, w_i)의 가중치) < $D[w_i]$
↔ D[천안] +(천안과 논산의 거리) < D[논산]
↔ $12 + 4 < \infty$ 이므로
$D[w_i]$= D[논산] 값을 에서 16으로 갱신

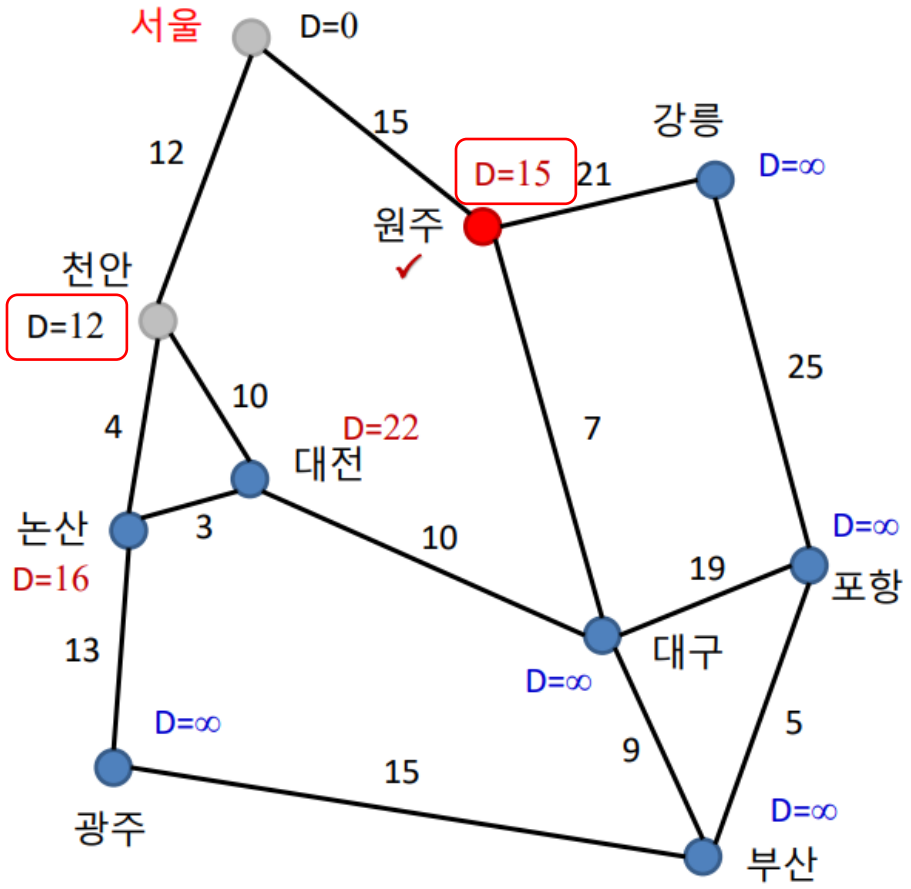


	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	∞	15	12	16	22	∞	∞	∞	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)

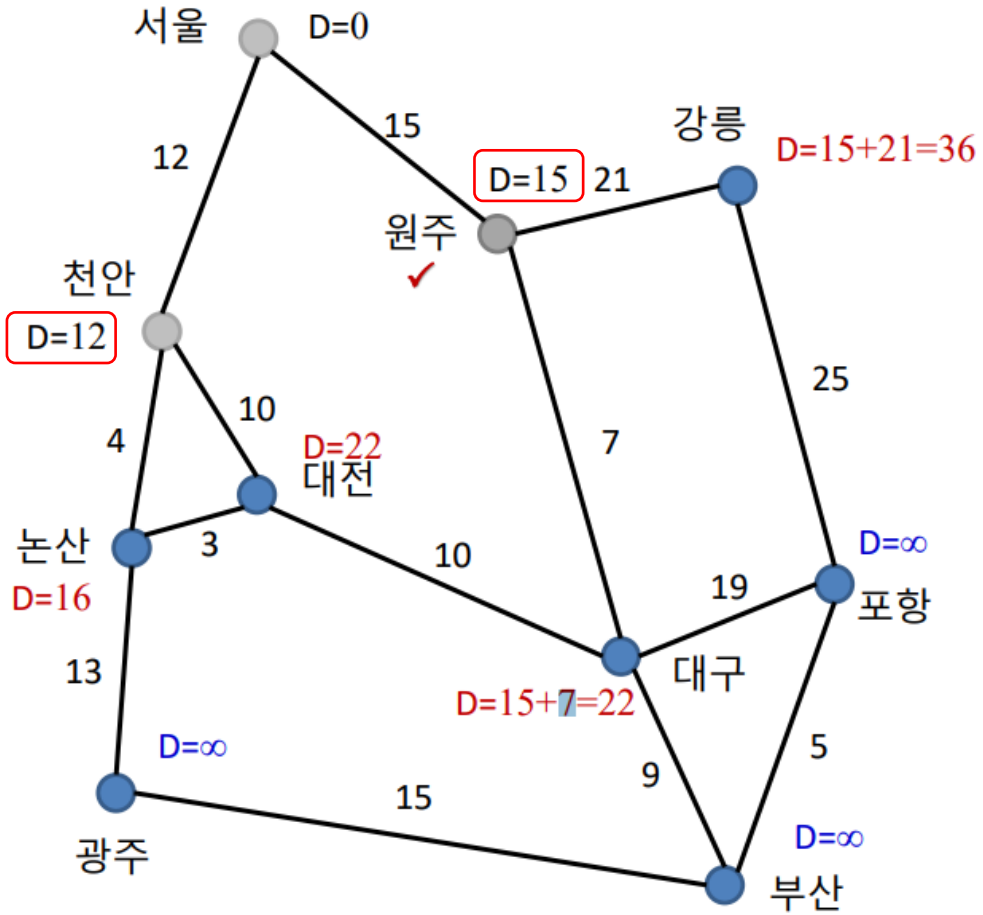


	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	22	22	∞	∞	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)

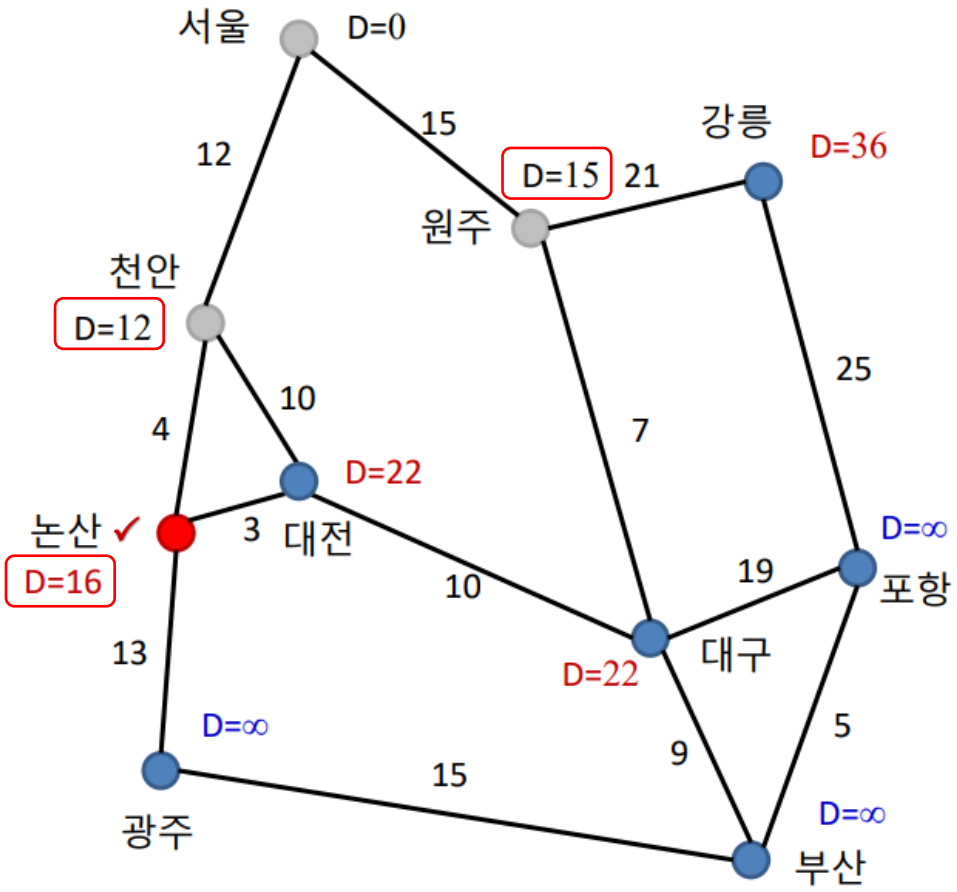


	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	22	22	∞	∞	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)

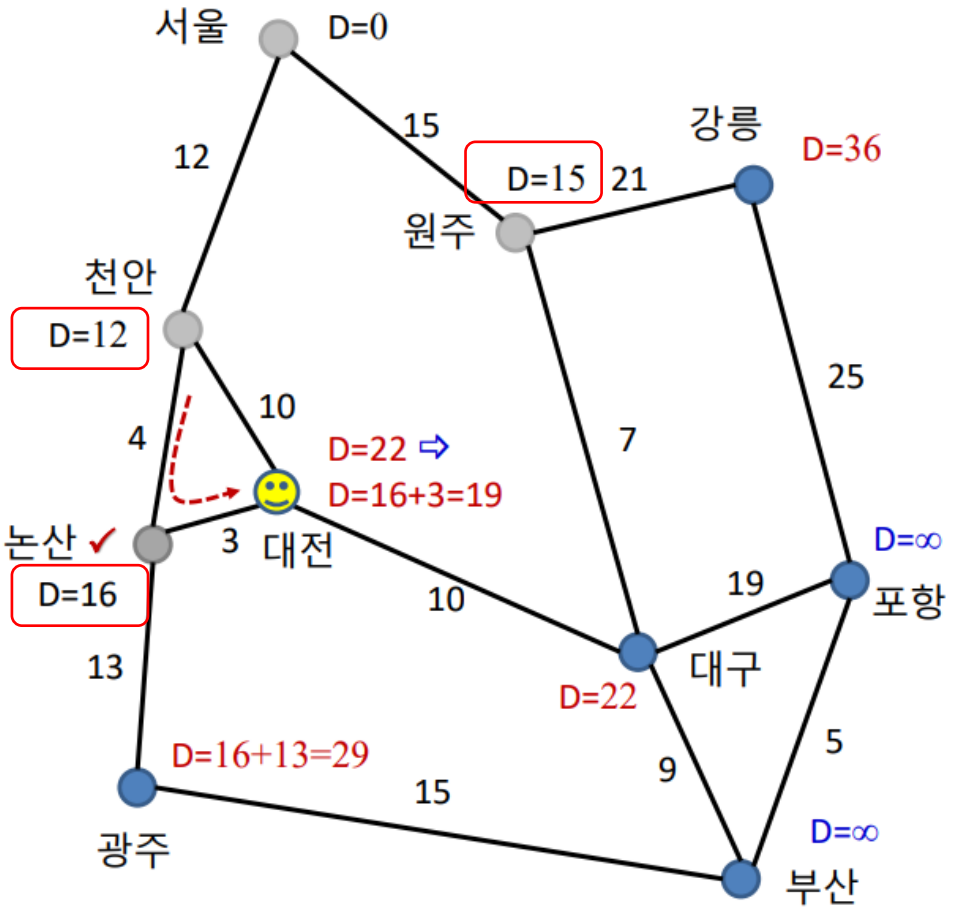


	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	19	22	∞	29	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)

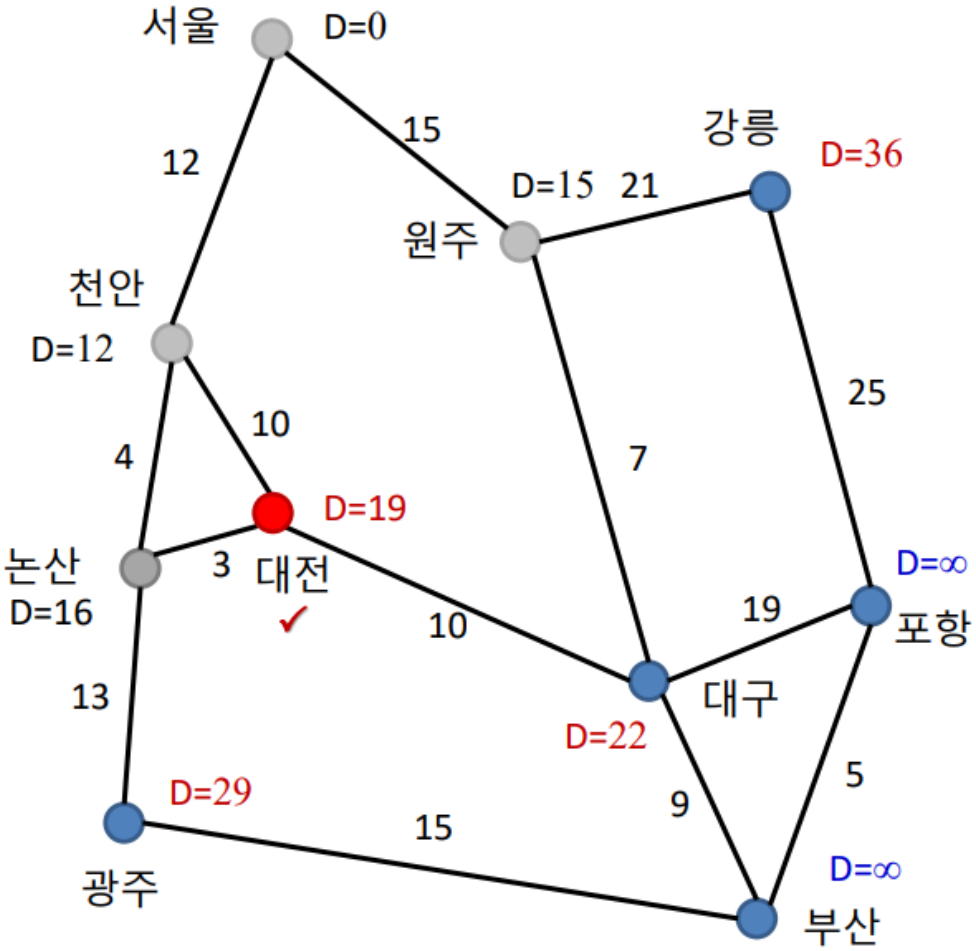


	서울	강릉	원주	천안	논산	대전	대구	포항	광주	부산
D = [	0	36	15	12	16	19	22	∞	29	∞]

4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

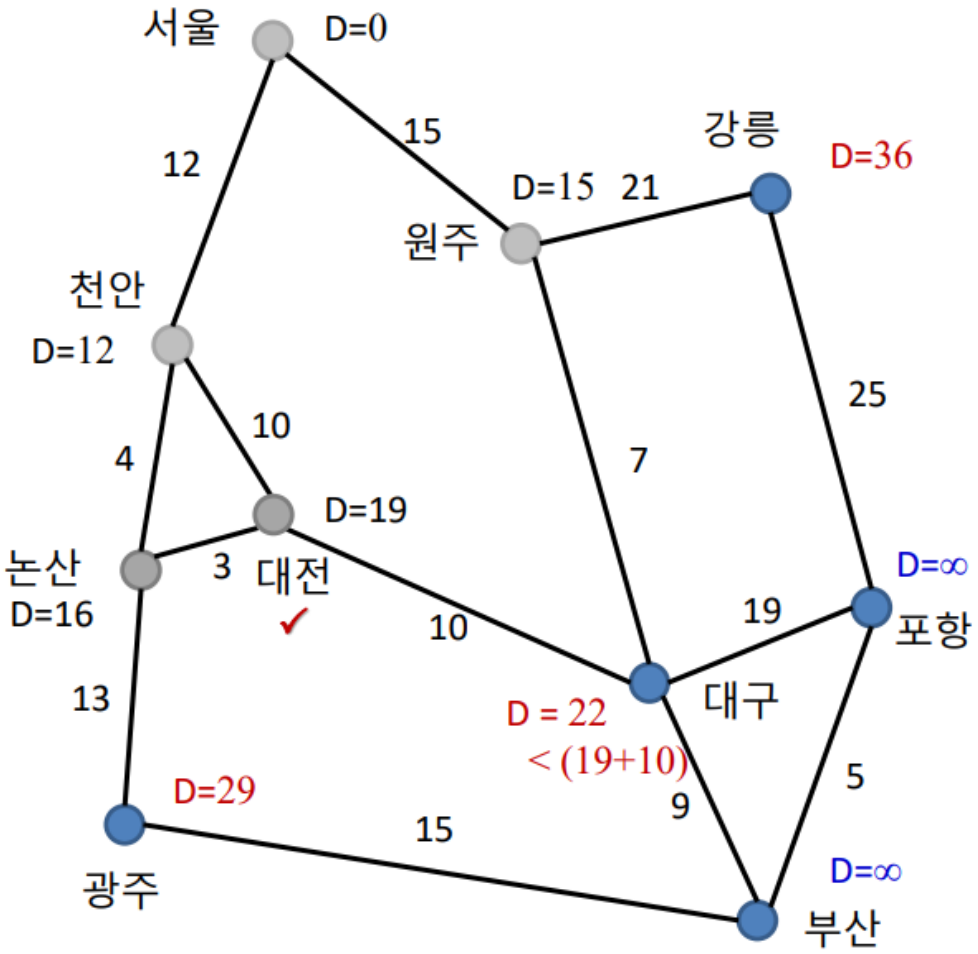
➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

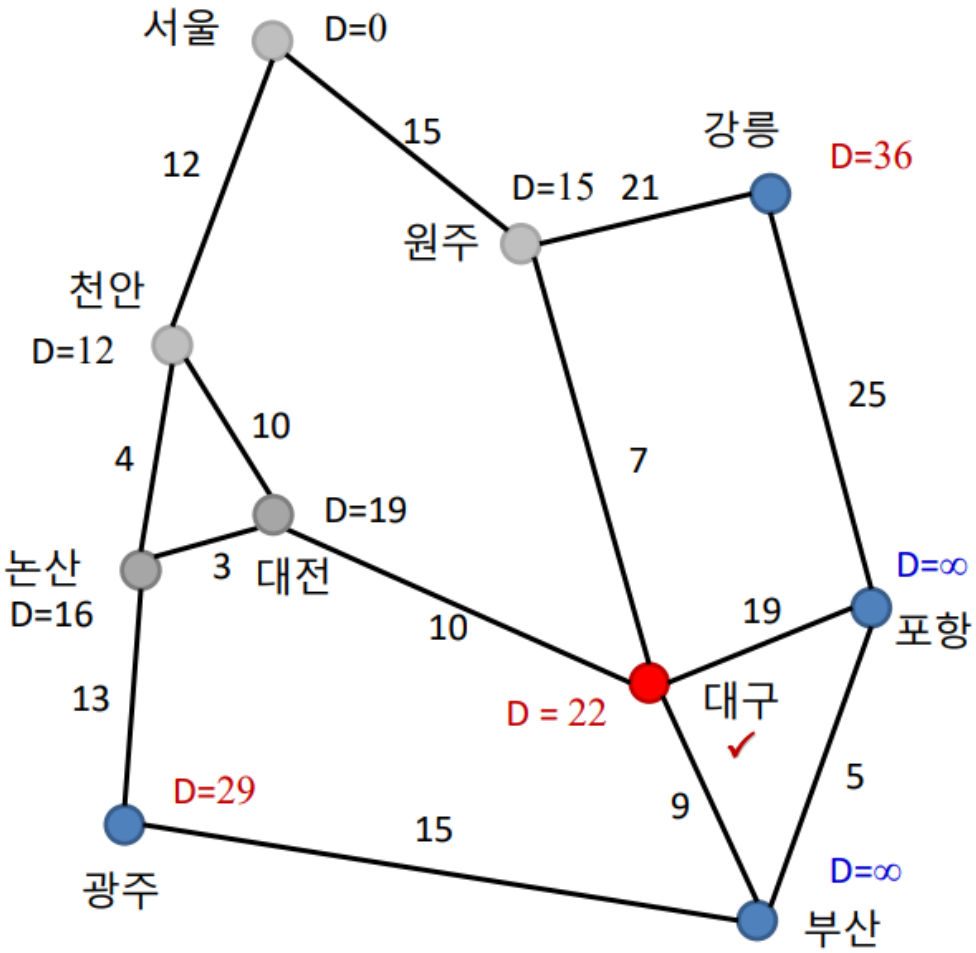
➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

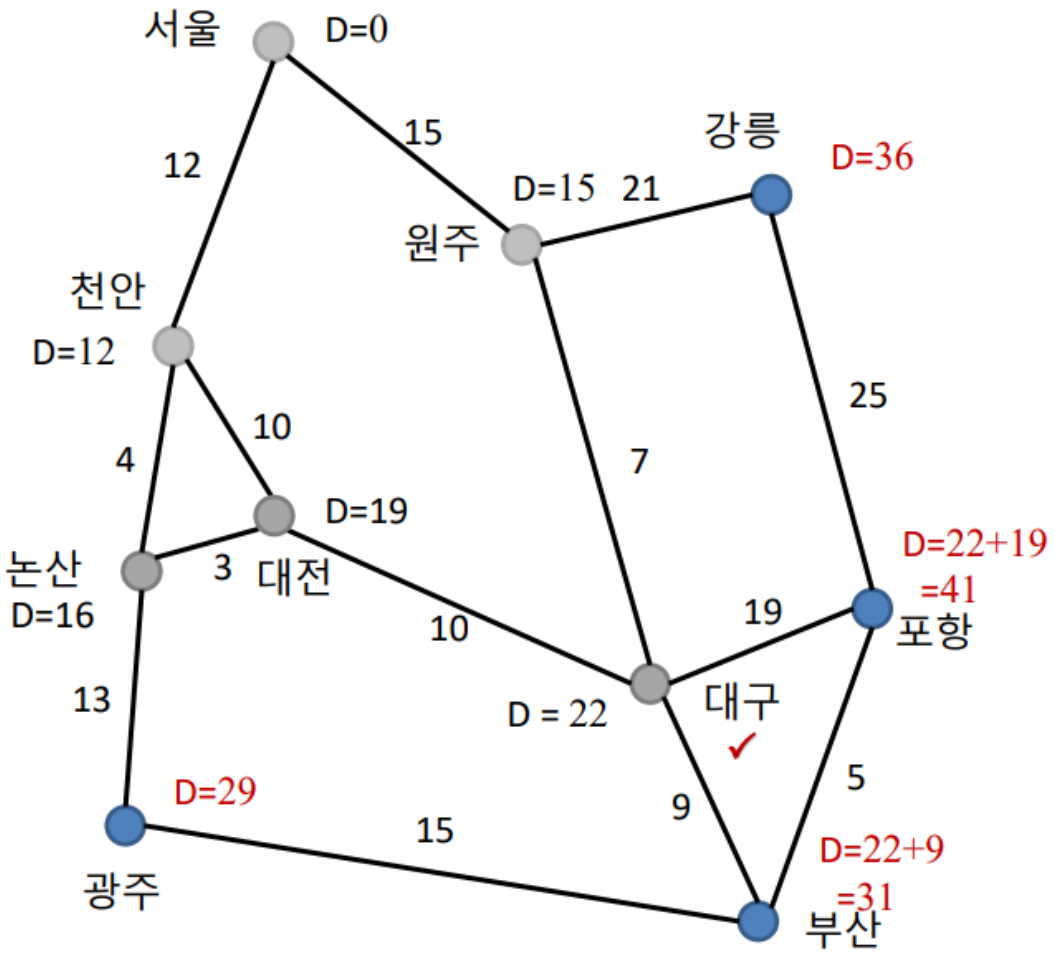
- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

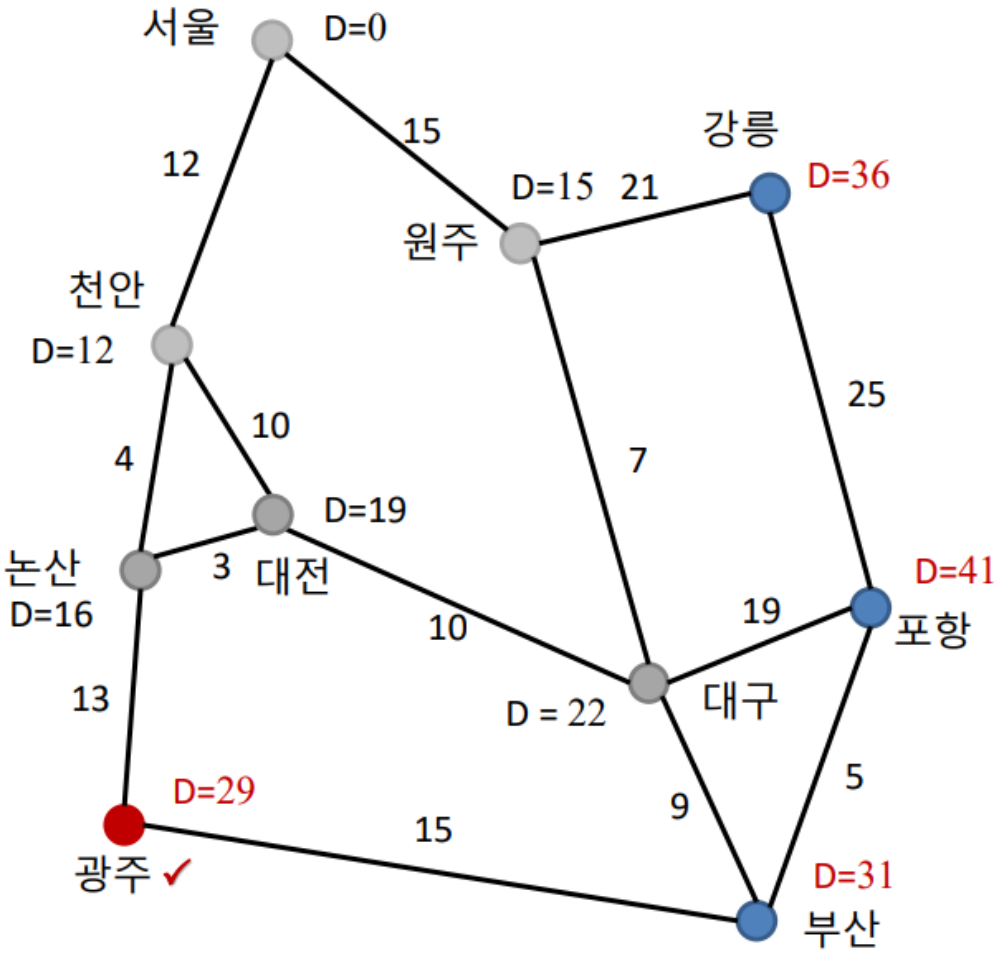
- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

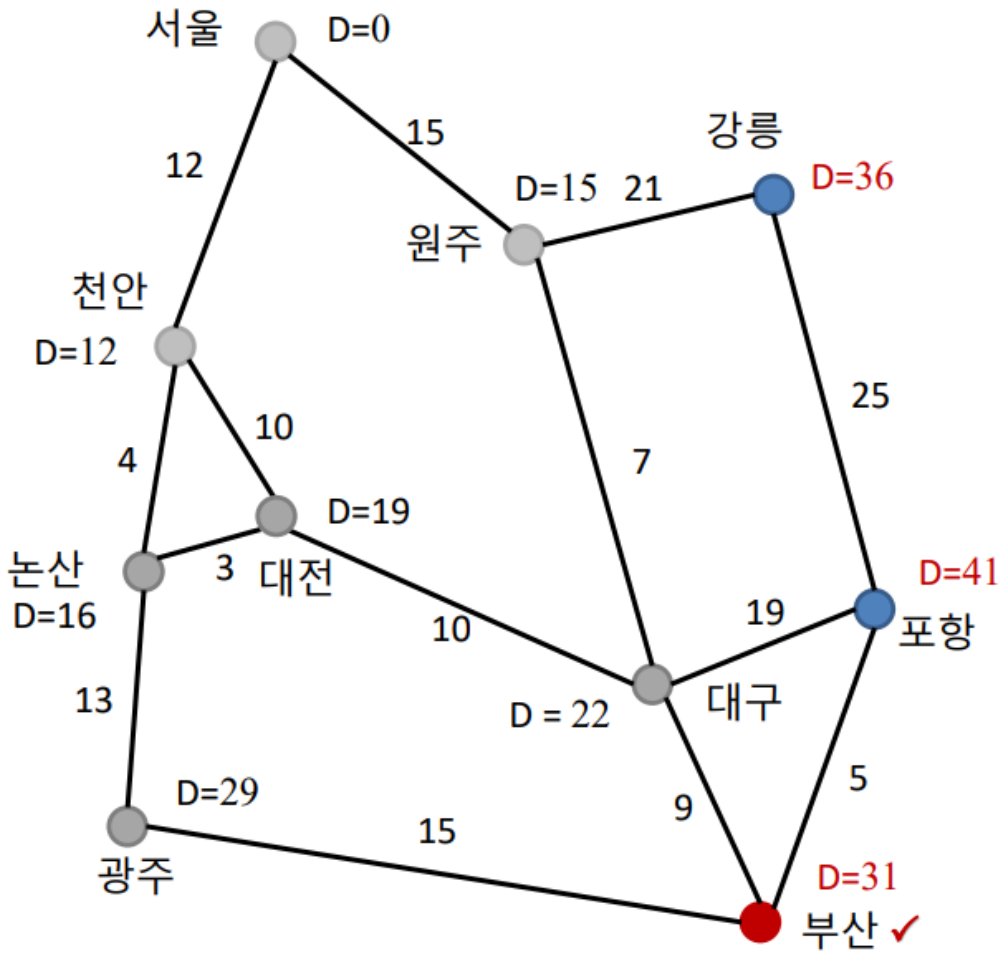
- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

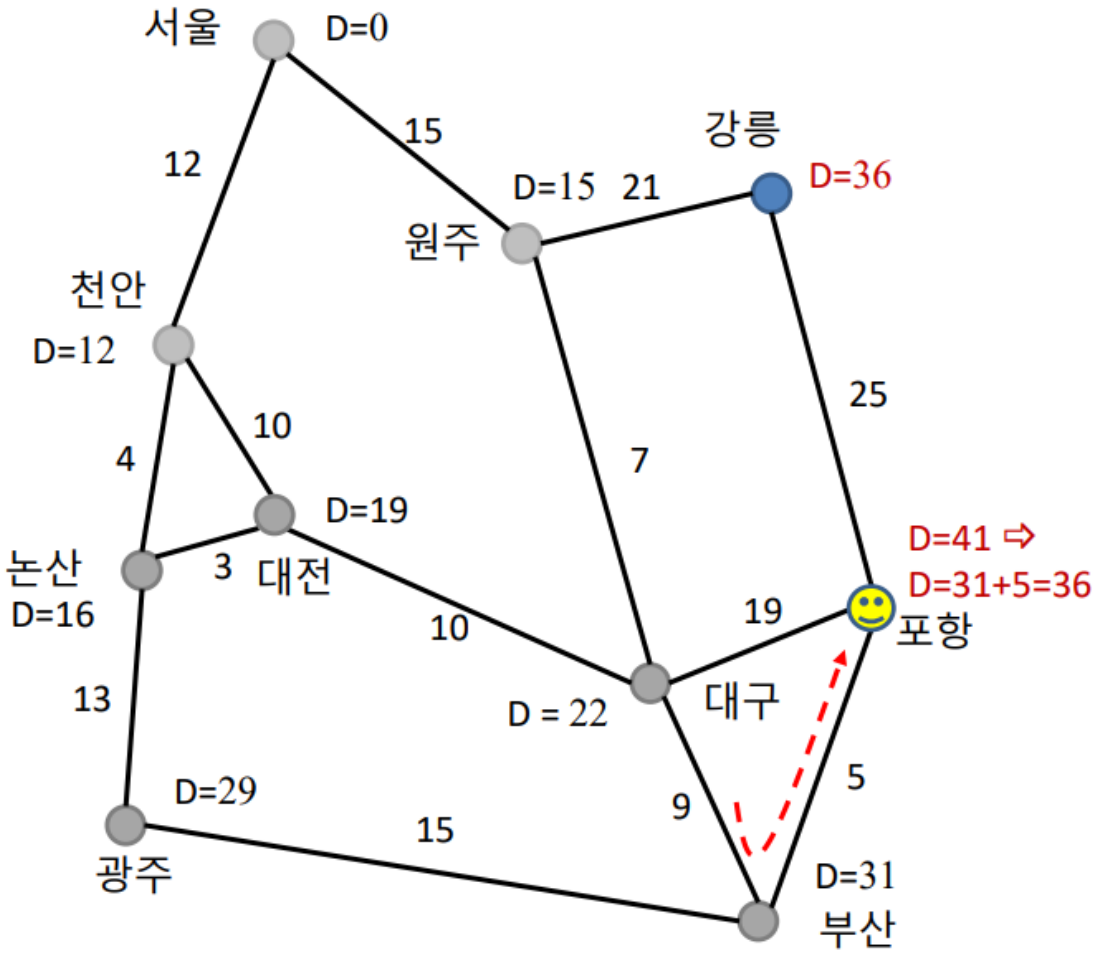
➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

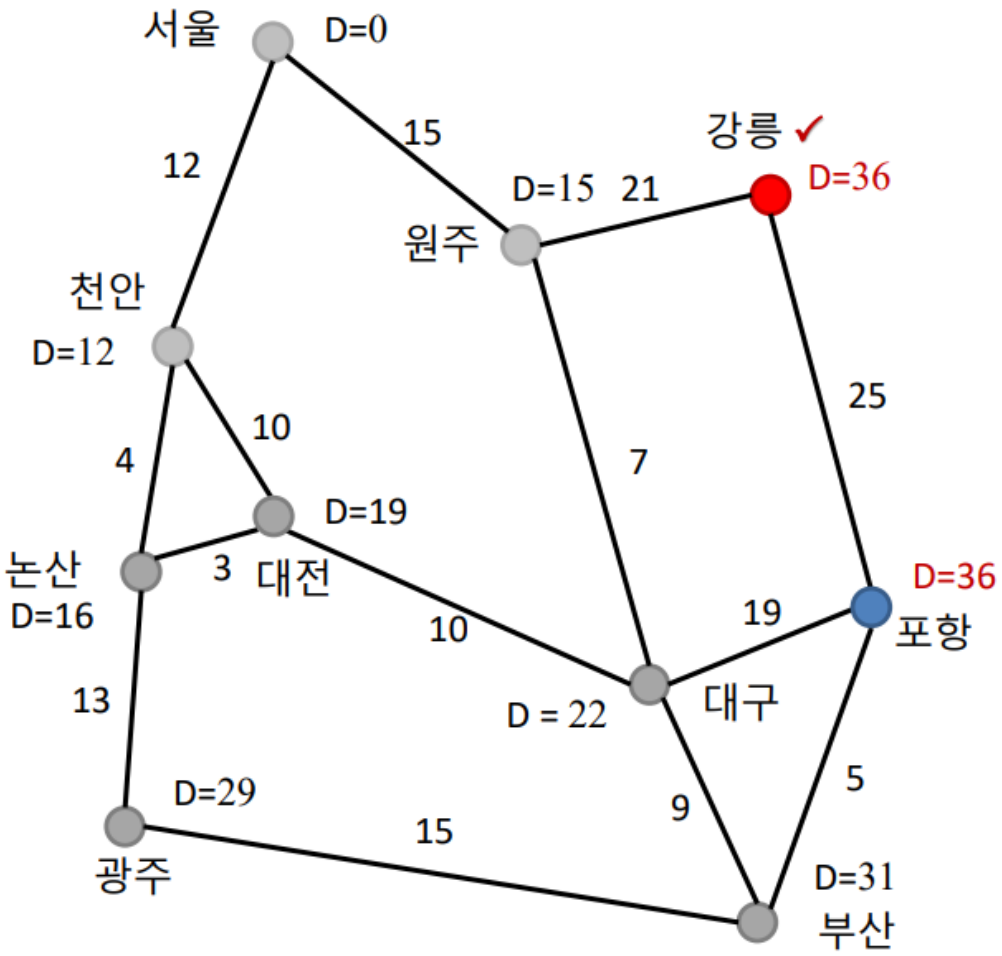
- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

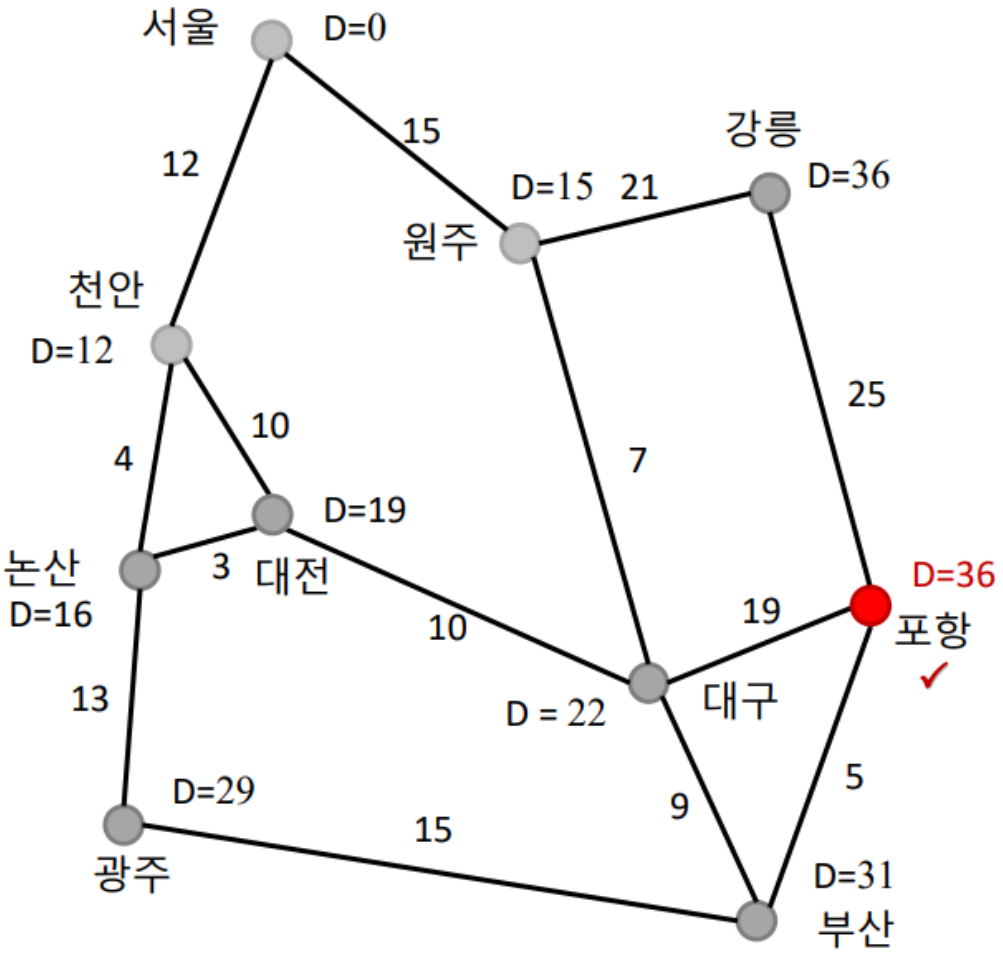
➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

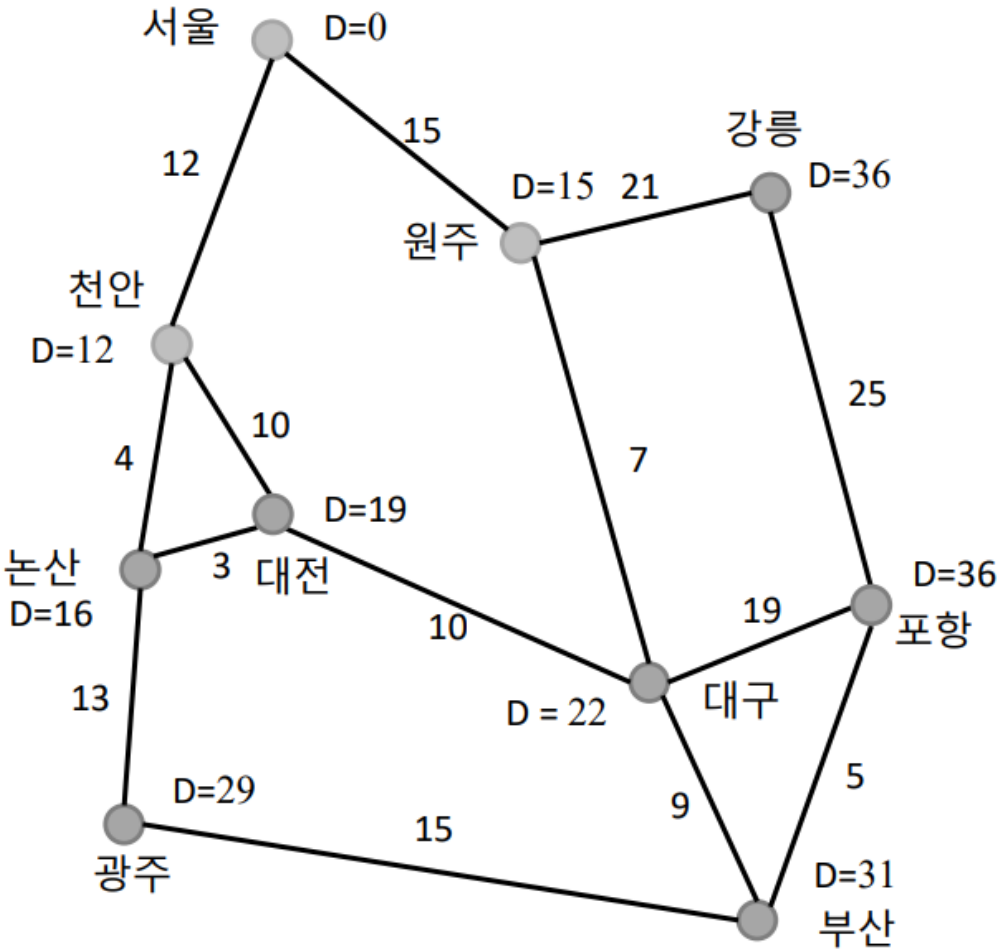
➢ ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



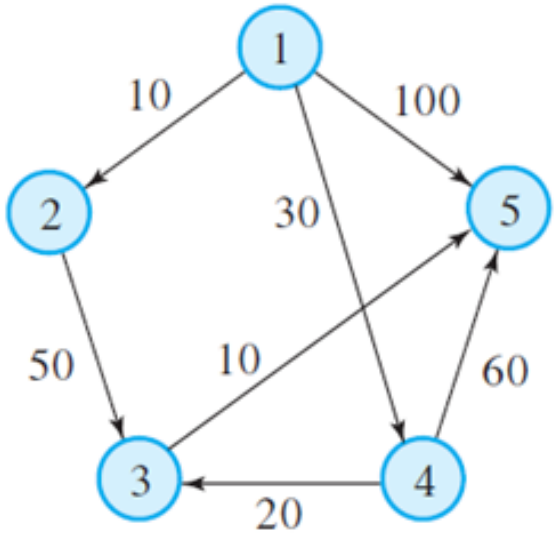
4.3 최단 경로 찾기 (Shortest Path)

❖다익스트라 알고리즘

- ShortestPath 알고리즘의 수행 과정 (출발점이 서울인 경우)



시작정점



같으며 4번째 반복 후 점 1에서 점 2, 점 3, 점 4
30, 60으로 최종 결정된다.

알고리즘 1 다익스트라 알고리즘

```
void Dijkstra()
    /* 이 알고리즘은 정점 1에서 방향 그래프의 모든 정점으로의 최단 거리를 계산한다.*/
{
    S = {1};   V ={2,3,4,5}
    for( i = 2 ; i <= n ; i++ )
        D[i] = C[1, i];  /* D값을 초기화한다 */
    for( i = 1 ; i <= n-1 ; i++ )
    {
        choose a vertex w in V - S such that
            D[w] is a minimum;
        add w to S;
        for(each vertex v in V - S)
            D[v] = min (D[v], D[w] + C[w, v]);
    }
} /* Dijkstra */
```

단계	S	w	$D[2]$	$D[3]$	$D[4]$	$D[5]$
0	{1}	—	10	∞	30	100
1	{1, 2}	2	10	60	30	100
2	{1, 2, 4}	4	10	50	30	90
3	{1, 2, 3, 4}	3	10	50	30	60
4	{1, 2, 3, 4, 5}	5	10	50	30	60

D[w]가 가장 작은 값을 갖는 w=4를 선택

4까지가는 최단거리를 알고 있을 때, 이 거리를 이용하여 다른 점들의 최단거리 갱신

4.3 최단 경로 찾기 (Shortest Path)

Cryptography & Information Security Laboratory

❖다익스트라 알고리즘

➢ 시간복잡도

– while-루프가 $(n-1)$번 반복되고,

- 1회 반복될 때 line 3에서 최소의 $D[v]$를 가진 점 v_{min}을 찾는데 $O(n)$ 시간이 걸린다. 왜냐하면 배열 D에서 최소값을 찾는 것이기 때문이다.
- 또한 line 4에서도 v_{min} 에 연결된 점의 수가 최대 $(n-1)$개이므로, 각 $D[w]$를 갱신하는데 걸리는 시간은 $O(n)$이다.

– 따라서 시간복잡도는 $(n-1)\,\mathrm{X}\,\{O(n)+O(n)\} = \boldsymbol{O(n^2)}$이다.

ShortestPath(G, s)

❖ 입력: 가중치 그래프 $G=(V,E), |V|=n, |E|=m$

❖ 출력: 출발점 s로부터 $(n-1)$개의 점까지 각각 최단 거리를 저장한 배열 D

1. 배열 D를 ∞로 초기화시킨다. 단, $D[s]=0$으로 초기화한다.
   // 배열 $D[v]$에는 출발점 s로부터 점 v까지의 거리가 저장된다.
   // 각 점 v에 대해서 $D[v]=\infty$로 초기화
2. *while* (s로부터의 최단 거리가 확정되지 않은 점이 있으면) { // $(n-1)$번 반복
3. 현재까지 s로부터 최단 거리가 확정되지 않은 각 점 v에 대해서 최소의 $D[v]$의 값을 가진 점 v_{min}을 선택하고, 출발점 s로부터 점 v_{min} 까지 의 최단 거리 $D[v_{min}]$을 확정 // $O(n)$
4. s로부터 현재보다 짧은 거리로 점 v_{min} 을 통해 우회 가능한 각 점 w에 대해서 $D[w]$를 갱신 // $O(n)$
   }
5. *return* D

4.3 최단 경로 찾기 (Shortest Path)

Cryptography & Information Security Laboratory

❖다익스트라 알고리즘

- 최단경로 찾기 알고리즘의 응용
  - 맵퀘스트 (Mapquest)와 구글 (Google) 웹사이트의 지도 서비스
  - 자동차 네비게이션
  - 네트워크와 통신 분야 및 모바일 네트워크
  - 산업 공학
  - 경영 공학의 운영 연구 (Operation Research)
  - 로봇 공학
  - 교통 공학
  - VLSI 디자인 분야 등